

[MS-ADFSPIP]:

Active Directory Federation Services and Proxy Integration Protocol

Intellectual Property Rights Notice for Open Specifications Documentation

- **Technical Documentation.** Microsoft publishes Open Specifications documentation ("this documentation") for protocols, file formats, data portability, computer languages, and standards support. Additionally, overview documents cover inter-protocol relationships and interactions.
- **Copyrights.** This documentation is covered by Microsoft copyrights. Regardless of any other terms that are contained in the terms of use for the Microsoft website that hosts this documentation, you can make copies of it in order to develop implementations of the technologies that are described in this documentation and can distribute portions of it in your implementations that use these technologies or in your documentation as necessary to properly document the implementation. You can also distribute in your implementation, with or without modification, any schemas, IDLs, or code samples that are included in the documentation. This permission also applies to any documents that are referenced in the Open Specifications documentation.
- **No Trade Secrets.** Microsoft does not claim any trade secret rights in this documentation.
- **Patents.** Microsoft has patents that might cover your implementations of the technologies described in the Open Specifications documentation. Neither this notice nor Microsoft's delivery of this documentation grants any licenses under those patents or any other Microsoft patents. However, a given Open Specifications document might be covered by the Microsoft [Open Specifications Promise](#) or the [Microsoft Community Promise](#). If you would prefer a written license, or if the technologies described in this documentation are not covered by the Open Specifications Promise or Community Promise, as applicable, patent licenses are available by contacting iplg@microsoft.com.
- **License Programs.** To see all of the protocols in scope under a specific license program and the associated patents, visit the [Patent Map](#).
- **Trademarks.** The names of companies and products contained in this documentation might be covered by trademarks or similar intellectual property rights. This notice does not grant any licenses under those rights. For a list of Microsoft trademarks, visit www.microsoft.com/trademarks.
- **Fictitious Names.** The example companies, organizations, products, domain names, email addresses, logos, people, places, and events that are depicted in this documentation are fictitious. No association with any real company, organization, product, domain name, email address, logo, person, place, or event is intended or should be inferred.

Reservation of Rights. All other rights are reserved, and this notice does not grant any rights other than as specifically described above, whether by implication, estoppel, or otherwise.

Tools. The Open Specifications documentation does not require the use of Microsoft programming tools or programming environments in order for you to develop an implementation. If you have access to Microsoft programming tools and environments, you are free to take advantage of them. Certain Open Specifications documents are intended for use in conjunction with publicly available standards specifications and network programming art and, as such, assume that the reader either is familiar with the aforementioned material or has immediate access to it.

Support. For questions and support, please contact dochelp@microsoft.com.

Revision Summary

Date	Revision History	Revision Class	Comments
8/8/2013	1.0	New	Released new document.
11/14/2013	2.0	Major	Significantly changed the technical content.
2/13/2014	3.0	Major	Significantly changed the technical content.
5/15/2014	3.0	None	No change to the meaning, language, or formatting of the technical content.
6/30/2015	4.0	Major	Significantly changed the technical content.
7/14/2016	5.0	Major	Significantly changed the technical content.
6/1/2017	5.0	None	No changes to the meaning, language, or formatting of the technical content.
9/15/2017	6.0	Major	Significantly changed the technical content.
9/12/2018	7.0	Major	Significantly changed the technical content.
4/7/2021	8.0	Major	Significantly changed the technical content.
4/23/2024	9.0	Major	Significantly changed the technical content.
9/16/2024	10.0	Major	Significantly changed the technical content.
11/19/2024	11.0	Major	Significantly changed the technical content.

Table of Contents

1	Introduction	10
1.1	Glossary	10
1.2	References	11
1.2.1	Normative References	11
1.2.2	Informative References	13
1.3	Overview	13
1.4	Relationship to Other Protocols	14
1.5	Prerequisites/Preconditions	15
1.6	Applicability Statement	15
1.7	Versioning and Capability Negotiation	15
1.8	Vendor-Extensible Fields	15
1.9	Standards Assignments.....	15
2	Messages.....	16
2.1	Transport	16
2.2	Common Data Types	16
2.2.1	HTTP Headers	16
2.2.1.1	X-MS-Proxy.....	16
2.2.1.2	X-MS-Forwarded-Client-IP.....	16
2.2.1.3	X-MS-Endpoint-Absolute-Path.....	17
2.2.1.4	X-MS-Target-Role	17
2.2.1.5	X-MS-ADFS-Proxy-Client-IP.....	17
2.2.1.6	X-MS-ProxyAuth-Token.....	17
2.2.2	Complex Types.....	18
2.2.2.1	Proxy Trust	18
2.2.2.2	Proxy Trust Renewal.....	18
2.2.2.3	Proxy Relying Party Trust	18
2.2.2.4	Configuration.....	18
2.2.2.5	Relying Party Trust List.....	20
2.2.2.6	Relying Party Trust.....	20
2.2.2.7	Relying Party Trust Publishing Settings	21
2.2.2.8	Store Entry List.....	21
2.2.2.9	Store Entry	22
2.2.2.10	Store Entry Key and Value	22
2.2.2.11	Serialized Request with Certificate	22
2.2.2.12	Port Type	23
2.2.2.13	Credential Collection Scheme	24
2.2.2.14	TLS Query Behavior	24
2.2.2.15	Certificate Validation	24
2.2.2.16	Certificate Type	25
2.2.2.17	Error Type.....	25
2.2.2.18	Proxy Token	25
2.2.2.19	Combined Token	26
2.2.2.20	Proxy Token Wrapper	26
2.2.2.21	Authentication Request.....	26
2.2.2.22	Error Response	27
3	Protocol Details.....	28
3.1	Common Details	28
3.1.1	Abstract Data Model.....	28
3.1.1.1	Server State.....	28
3.1.1.2	Client State.....	28
3.1.1.3	Relying Party Trust State	29
3.1.2	Timers	29
3.1.3	Initialization.....	29

3.1.4	Higher-Layer Triggered Events	29
3.1.5	Message Processing Events and Sequencing Rules	29
3.1.6	Timer Events.....	29
3.1.7	Other Local Events.....	29
3.2	Proxy Registration Server Details	29
3.2.1	Abstract Data Model.....	29
3.2.2	Timers	30
3.2.3	Initialization.....	30
3.2.4	Higher-Layer Triggered Events	30
3.2.5	Message Processing Events and Sequencing Rules	30
3.2.5.1	Proxy/EstablishTrust.....	30
3.2.5.1.1	POST	30
3.2.5.1.1.1	Request Body	31
3.2.5.1.1.2	Response Body	31
3.2.5.1.1.3	Processing Details.....	31
3.2.5.2	Proxy/RenewTrust.....	31
3.2.5.2.1	POST	31
3.2.5.2.1.1	Request Body	32
3.2.5.2.1.2	Response Body	32
3.2.5.2.1.3	Processing Details.....	32
3.2.5.3	Proxy/WebApplicationProxy/Trust	32
3.2.5.3.1	GET	32
3.2.5.3.1.1	Request Body	33
3.2.5.3.1.2	Response Body	33
3.2.5.3.1.3	Processing Details.....	33
3.2.5.3.2	POST	33
3.2.5.3.2.1	Request Body	34
3.2.5.3.2.2	Response Body	34
3.2.5.3.2.3	Processing Details.....	34
3.2.5.3.3	DELETE	34
3.2.5.3.3.1	Request Body	35
3.2.5.3.3.2	Response Body	35
3.2.5.3.3.3	Processing Details.....	35
3.2.6	Timer Events.....	35
3.2.7	Other Local Events.....	35
3.3	Proxy Registration Client Details	35
3.3.1	Abstract Data Model.....	35
3.3.2	Timers	35
3.3.3	Initialization.....	35
3.3.4	Higher-Layer Triggered Events	35
3.3.5	Message Processing Events and Sequencing Rules	35
3.3.5.1	Proxy/EstablishTrust.....	36
3.3.5.1.1	POST	36
3.3.5.1.1.1	Request Body	36
3.3.5.1.1.2	Response Body	36
3.3.5.1.1.3	Processing Details.....	36
3.3.5.2	Proxy/RenewTrust.....	36
3.3.5.2.1	POST	36
3.3.5.2.1.1	Request Body	36
3.3.5.2.1.2	Response Body	36
3.3.5.2.1.3	Processing Details.....	36
3.3.5.3	Proxy/WebApplicationProxy/Trust	36
3.3.5.3.1	GET	36
3.3.5.3.1.1	Request Body	37
3.3.5.3.1.2	Response Body	37
3.3.5.3.1.3	Processing Details.....	37
3.3.5.3.2	POST	37
3.3.5.3.2.1	Request Body	37

3.3.5.3.2.2	Response Body	37
3.3.5.3.2.3	Processing Details	37
3.3.5.3.3	DELETE	37
3.3.5.3.3.1	Request Body	37
3.3.5.3.3.2	Response Body	37
3.3.5.3.3.3	Processing Details	37
3.3.6	Timer Events	37
3.3.7	Other Local Events	37
3.4	Service Configuration Server Details	38
3.4.1	Abstract Data Model	38
3.4.2	Timers	38
3.4.3	Initialization	38
3.4.4	High-Layer Triggered Events	38
3.4.5	Message Processing Events and Sequencing Rules	38
3.4.5.1	Proxy/GetConfiguration	39
3.4.5.1.1	GET	39
3.4.5.1.1.1	Request Body	39
3.4.5.1.1.2	Response Body	39
3.4.5.1.1.3	Processing Details	39
3.4.5.2	Proxy/RelyingPartyTrusts	39
3.4.5.2.1	GET	39
3.4.5.2.1.1	Request Body	40
3.4.5.2.1.2	Response Body	40
3.4.5.2.1.3	Processing Details	40
3.4.5.3	Proxy/RelyingPartyTrusts/	40
3.4.5.3.1	GET	40
3.4.5.3.1.1	Request Body	41
3.4.5.3.1.2	Response Body	41
3.4.5.3.1.3	Processing Details	41
3.4.6	Timer Events	41
3.4.7	Other Local Events	41
3.5	Service Configuration Client Details	41
3.5.1	Abstract Data Model	41
3.5.2	Timers	41
3.5.3	Initialization	41
3.5.4	High-Layer Triggered Events	41
3.5.5	Message Processing Events and Sequencing Rules	42
3.5.5.1	Proxy/GetConfiguration	42
3.5.5.1.1	GET	42
3.5.5.1.1.1	Request Body	42
3.5.5.1.1.2	Response Body	42
3.5.5.1.1.3	Processing Details	42
3.5.5.2	Proxy/RelyingPartyTrusts	42
3.5.5.2.1	GET	42
3.5.5.2.1.1	Request Body	42
3.5.5.2.1.2	Response Body	42
3.5.5.2.1.3	Processing Details	42
3.5.5.3	Proxy/RelyingPartyTrusts/	42
3.5.5.3.1	GET	42
3.5.5.3.1.1	Request Body	43
3.5.5.3.1.2	Response Body	43
3.5.5.3.1.3	Processing Details	43
3.5.6	Timer Events	43
3.5.7	Other Local Events	43
3.6	Proxy Configuration Server Details	43
3.6.1	Abstract Data Model	43
3.6.2	Timers	43
3.6.3	Initialization	43

3.6.4	High-Layer Triggered Events	43
3.6.5	Message Processing Events and Sequencing Rules	43
3.6.5.1	Proxy/WebApplicationProxy/Store	44
3.6.5.1.1	GET	44
3.6.5.1.1.1	Request Body	44
3.6.5.1.1.2	Response Body	45
3.6.5.1.1.3	Processing Details	45
3.6.5.2	Proxy/WebApplicationProxy/Store/	45
3.6.5.2.1	GET	45
3.6.5.2.1.1	Request Body	45
3.6.5.2.1.2	Response Body	45
3.6.5.2.1.3	Processing Details	45
3.6.5.2.2	POST	46
3.6.5.2.2.1	Request Body	46
3.6.5.2.2.2	Response Body	46
3.6.5.2.2.3	Processing Details	46
3.6.5.2.3	PUT	46
3.6.5.2.3.1	Request Body	47
3.6.5.2.3.2	Response Body	47
3.6.5.2.3.3	Processing Details	47
3.6.5.2.4	DELETE	47
3.6.5.2.4.1	Request Body	48
3.6.5.2.4.2	Response Body	48
3.6.5.2.4.3	Processing Details	48
3.6.6	Timer Events	48
3.6.7	Other Local Events	48
3.7	Proxy Configuration Client Details	49
3.7.1	Abstract Data Model	49
3.7.2	Timers	49
3.7.3	Initialization	49
3.7.4	High-Layer Triggered Events	49
3.7.5	Message Processing Events and Sequencing Rules	49
3.7.5.1	Proxy/WebApplicationProxy/Store	49
3.7.5.1.1	GET	49
3.7.5.1.1.1	Response Body	49
3.7.5.1.1.2	Request Body	49
3.7.5.1.1.3	Processing Details	49
3.7.5.2	Proxy/WebApplicationProxy/Store/	49
3.7.5.2.1	GET	49
3.7.5.2.1.1	Request Body	49
3.7.5.2.1.2	Response Body	50
3.7.5.2.1.3	Processing Details	50
3.7.5.2.2	POST	50
3.7.5.2.2.1	Request Body	50
3.7.5.2.2.2	Response Body	50
3.7.5.2.2.3	Processing Details	50
3.7.5.2.3	PUT	50
3.7.5.2.3.1	Request Body	50
3.7.5.2.3.2	Response Body	50
3.7.5.2.3.3	Processing Details	50
3.7.5.2.4	DELETE	50
3.7.5.2.4.1	Request Body	50
3.7.5.2.4.2	Response Body	50
3.7.5.2.4.3	Processing Details	50
3.7.6	Timer Events	51
3.7.7	Other Local Events	51
3.8	Application Publishing Server Details	51
3.8.1	Abstract Data Model	51

3.8.2	Timers	51
3.8.3	Initialization	51
3.8.4	High-Layer Triggered Events	51
3.8.5	Message Processing Events and Sequencing Rules	51
3.8.5.1	Proxy/RelyingPartyTrusts/{Identifier}/PublishedSettings	52
3.8.5.1.1	POST	52
3.8.5.1.1.1	Request Body	52
3.8.5.1.1.2	Response Body	52
3.8.5.1.1.3	Processing Details	52
3.8.5.1.2	DELETE	53
3.8.5.1.2.1	Request Body	53
3.8.5.1.2.2	Response Body	53
3.8.5.1.2.3	Processing Details	53
3.8.6	Timer Events	54
3.8.7	Other Local Events	54
3.9	Application Publishing Client Details	54
3.9.1	Abstract Data Model	54
3.9.2	Timers	54
3.9.3	Initialization	54
3.9.4	High-Layer Triggered Events	54
3.9.5	Message Processing Events and Sequencing Rules	54
3.9.5.1	Proxy/RelyingPartyTrusts/{Identifier}/PublishedSettings	55
3.9.5.1.1	POST	55
3.9.5.1.1.1	Request Body	55
3.9.5.1.1.2	Response Body	55
3.9.5.1.1.3	Processing Details	55
3.9.5.1.2	DELETE	55
3.9.5.1.2.1	Request Body	55
3.9.5.1.2.2	Response Body	55
3.9.5.1.2.3	Processing Details	55
3.9.6	Timer Events	55
3.9.7	Other Local Events	55
3.10	Proxy Runtime Behaviors Server Details	56
3.10.1	Abstract Data Model	56
3.10.2	Timers	56
3.10.3	Initialization	56
3.10.4	High-Layer Triggered Events	56
3.10.5	Message Processing Events and Sequencing Rules	56
3.10.5.1	BackEndProxyTLS	56
3.10.5.1.1	POST	56
3.10.5.1.1.1	Request Body	57
3.10.5.1.1.2	Response Body	57
3.10.5.1.1.3	Processing Details	57
3.10.6	Timer Events	57
3.10.7	Other Local Events	57
3.11	Proxy Runtime Behaviors Client Details	57
3.11.1	Abstract Data Model	57
3.11.2	Timers	57
3.11.3	Initialization	58
3.11.4	High-Layer Triggered Events	58
3.11.5	Message Processing Events and Sequencing Rules	58
3.11.5.1	End-user X509 Certificate Success Processing	59
3.11.5.2	End-user X509 Certificate Common Processing	60
3.11.6	Timer Events	60
3.11.7	Other Local Events	60
3.12	Application Proxy Runtime Behaviors Server Details	60
3.12.1	Abstract Data Model	60
3.12.2	Timers	60

3.12.3	Initialization	60
3.12.4	High-Layer Triggered Events	60
3.12.5	Message Processing Events and Sequencing Rules	61
3.12.5.1	Issue Preauthentication	61
3.12.5.1.1	Proxy Preauthentication	61
3.12.5.1.1.1	Request Body	61
3.12.5.1.1.2	Response Body	61
3.12.5.1.1.3	Processing Details	61
3.12.5.1.2	SAML-P Extensions for Preauthentication	62
3.12.5.1.3	WS-Fed Extensions for Preauthentication	62
3.12.5.1.4	OAuth Extensions for Preauthentication	63
3.12.5.1.5	Proxy Preauthentication for Active Clients	63
3.12.5.1.5.1	Request Body	63
3.12.5.1.5.2	Response Body	64
3.12.5.1.5.3	Processing Details	64
3.12.6	Timer Events	64
3.12.7	Other Local Events	64
3.13	Application Proxy Runtime Behaviors Client Details	64
3.13.1	Abstract Data Model	64
3.13.2	Timers	64
3.13.3	Initialization	64
3.13.4	High-Layer Triggered Events	64
3.13.5	Message Processing Events and Sequencing Rules	65
3.13.5.1	Preauthentication	65
3.13.5.1.1	Query String Based Preauthentication	65
3.13.5.1.2	HTTP Authorization Header Based Preauthentication	65
3.13.5.2	Initiate Preauthentication	65
3.13.5.2.1	Initiate Redirect-based Preauthentication	65
3.13.5.2.2	Response to [MS-OFBA] Requests	66
3.13.5.2.3	Response to Active Requests	67
3.13.6	Timer Events	68
3.13.7	Other Local Events	68
4	Protocol Examples	69
4.1	Establishing Proxy Trust with the Server	69
4.1.1	Client Request	69
4.1.2	Server Response	69
4.2	Getting Information about All Relying Party Trusts	69
4.2.1	Client Request	69
4.2.2	Server Response	69
4.3	Create a New Set of Published Settings on a Relying Party Trust	70
4.3.1	Client Request	70
4.3.2	Server Response	70
4.4	Remove an Existing Set of Published Settings on a Relying Party Trust	70
4.4.1	Client Request	70
4.4.2	Server Response	70
4.5	Add a Key Value Pair to the Store	70
4.5.1	Client Request	70
4.5.2	Server Response	71
4.6	Retrieve a Value of a Key from the Store	71
4.6.1	Client Request	71
4.6.2	Server Response	71
4.7	Update the Value of a Key Already in the Store	71
4.7.1	Client Request	71
4.7.2	Server Response	71
4.8	Create a new Proxy Relying Party Trust	72
4.8.1	Client Request	72
4.8.2	Server Response	72

4.9	Get the Proxy Relying Party Trust.....	72
4.9.1	Client Request.....	72
4.9.2	Server Response	72
5	Security	73
5.1	Security Considerations for Implementers	73
5.2	Index of Security Parameters	73
6	Appendix A: Full JSON Schema	74
7	Appendix B: Product Behavior	79
8	Change Tracking.....	81
9	Index.....	82

1 Introduction

The Active Directory Federation Services (AD FS) Proxy and Web Application Proxy Integration Protocol (ADSFPIP) define the interactions between AD FS Proxy and the Web Application Proxy that allows on-premises applications to be published to the Internet and be pre-authenticated against the AD FS. The intended functionality of the system and how the protocols in this system interact are described.

Sections 1.5, 1.8, 1.9, 2, and 3 of this specification are normative. All other sections and examples in this specification are informative.

1.1 Glossary

This document uses the following terms:

Active Directory Federation Services (AD FS): A Microsoft implementation of a federation services provider, which provides a security token service (STS) that can issue security tokens to a caller using various protocols such as WS-Trust, WS-Federation, and Security Assertion Markup Language (SAML) version 2.0.

Active Directory Federation Services (AD FS) farm: A collection of AD FS servers that is typically maintained by an enterprise to obtain greater redundancy and offer more reliable service than a single standalone AD FS server.

Active Directory Federation Services and Proxy system: A system of features and protocols whereby a client located outside the boundaries of a corporate network can access application services located inside those boundaries.

Coordinated Universal Time (UTC): A high-precision atomic time standard that approximately tracks Universal Time (UT). It is the basis for legal, civil time all over the Earth. Time zones around the world are expressed as positive and negative offsets from UTC. In this role, it is also referred to as Zulu time (Z) and Greenwich Mean Time (GMT). In these specifications, all references to UTC refer to the time at UTC-0 (or GMT).

extended key usage (EKU): An X.509 certificate extension that indicates one or more purposes for which the certificate can be used.

farm configuration: A collection of servers, each of which provide the same services, and to each of which a service request can be routed for load balancing.

Hypertext Transfer Protocol (HTTP): An application-level protocol for distributed, collaborative, hypermedia information systems (text, graphic images, sound, video, and other multimedia files) on the World Wide Web.

Hypertext Transfer Protocol Secure (HTTPS): An extension of HTTP that securely encrypts and decrypts web page requests. In some older protocols, "Hypertext Transfer Protocol over Secure Sockets Layer" is still used (Secure Sockets Layer has been deprecated). For more information, see [\[SSL3\]](#) and [\[RFC5246\]](#).

internal network: The portion of the corporate network that is protected by a firewall.

JavaScript Object Notation (JSON): A text-based, data interchange format that is used to transmit structured data, typically in Asynchronous JavaScript + XML (AJAX) web applications, as described in [\[RFC7159\]](#). The JSON format is based on the structure of ECMAScript (Jscript, JavaScript) objects.

JSON Web Token (JWT): A string representing a set of claims as a JSON object that is encoded in a JWS or JWE, enabling the claims to be digitally signed or integrity protected with a Message Authentication Code (MAC) and/or encrypted. For more information, see [\[RFC7519\]](#).

non-claims-aware: A characteristic of a network device or application that makes it unable to participate in claims-based authentication.

perimeter network: The portion of the corporate network that is on the outside of the firewall and is exposed to external network traffic.

pre-authentication: In **Active Directory Federation Services (AD FS)**, the act of enforcing authentication of a user on the edge of a protected network boundary.

proxy: A network node that accepts network traffic originating from one network agent and transmits it to another network agent.

token: A set of rights and privileges for a given user.

Transport Layer Security (TLS): A security protocol that supports confidentiality and integrity of messages in client and server applications communicating over open networks. TLS supports server and, optionally, client authentication by using X.509 certificates (as specified in [\[X509\]](#)). TLS is standardized in the IETF TLS working group.

Web Application Proxy: A set of components that provide proxy services for clients that are requesting access to application services inside the boundaries of a corporate network.

MAY, SHOULD, MUST, SHOULD NOT, MUST NOT: These terms (in all caps) are used as defined in [\[RFC2119\]](#). All statements of optional behavior use either MAY, SHOULD, or SHOULD NOT.

1.2 References

Links to a document in the Microsoft Open Specifications library point to the correct section in the most recently published version of the referenced document. However, because individual documents in the library are not updated at the same time, the section numbers in the documents may not match. You can confirm the correct section numbering by checking the [Errata](#).

1.2.1 Normative References

We conduct frequent surveys of the normative references to assure their continued availability. If you have any issue with finding a normative reference, please contact dochelp@microsoft.com. We will assist you in finding the relevant information.

[IETFDRAFT-JWS] Internet Engineering Task Force (IETF), "JSON Web Signature (JWS)", draft-ietf-jose-json-web-signature-10, April 2013, <http://tools.ietf.org/html/draft-ietf-jose-json-web-signature-10>

[IETFDRAFT-TOKBIND-H] Popov, A., Ed., Nystroem, M., and Balfanz, D., "Token Binding over HTTP", draft-ietf-tokbind-https-18, June 2018, <https://tools.ietf.org/html/draft-ietf-tokbind-https-18>

[IETFDRAFT-TOKBIND-T] Campbell, B., "HTTPS Token Binding with TLS Terminating Reverse Proxies", draft-ietf-tokbind-ttrp-06, July 2018, <https://tools.ietf.org/html/draft-ietf-tokbind-ttrp-06>

[IETFDRAFT-TOKBINDPROT] Popov, A., Ed., Nystroem, M., Balfanz, D., et al., "The Token Binding Protocol Version 1.0", draft-ietf-tokbind-protocol-19, May 2018, <https://tools.ietf.org/html/draft-ietf-tokbind-protocol-19>

[IETFDRAFT-TOKBND] Balfanz, D., Langley, A., Nystroem, M., et al., "Transport Layer Security (TLS) Extension for Token Binding Protocol Negotiation", draft-popov-tokbind-negotiation-00, May 2015, <https://datatracker.ietf.org/doc/html/draft-popov-tokbind-negotiation>

[MS-OAPX] Microsoft Corporation, "[OAuth 2.0 Protocol Extensions](#)".

[MS-OFBA] Microsoft Corporation, "[Office Forms Based Authentication Protocol](#)".

[MS-PKAP] Microsoft Corporation, "[Public Key Authentication Protocol](#)".

[MSKB-4034661] Microsoft Corporation, "August 16, 2017 - KB4034661 (OS Build 14393.1613)", <https://support.microsoft.com/help/4034661>

[RFC1422] Kent, S., "Privacy Enhancement for Internet Electronic Mail: Part II: Certificate-Based Key Management", RFC 1422, February 1993, <https://www.rfc-editor.org/info/rfc1422>

[RFC1738] Berners-Lee, T., Masinter, L., and McCahill, M., Eds., "Uniform Resource Locators (URL)", RFC 1738, December 1994, <https://www.rfc-editor.org/info/rfc1738>

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>

[RFC2246] Dierks, T., and Allen, C., "The TLS Protocol Version 1.0", RFC 2246, January 1999, <https://www.rfc-editor.org/info/rfc2246>

[RFC2478] Baize, E. and Pinkas, D., "The Simple and Protected GSS-API Negotiation Mechanism", RFC 2478, December 1998, <https://www.rfc-editor.org/info/rfc2478>

[RFC2616] Fielding, R., Gettys, J., Mogul, J., et al., "Hypertext Transfer Protocol -- HTTP/1.1", RFC 2616, June 1999, <https://www.rfc-editor.org/info/rfc2616>

[RFC2617] Franks, J., Hallam-Baker, P., Hostetler, J., et al., "HTTP Authentication: Basic and Digest Access Authentication", RFC 2617, June 1999, <https://www.rfc-editor.org/info/rfc2617>

[RFC2818] Rescorla, E., "HTTP Over TLS", RFC 2818, May 2000, <https://www.rfc-editor.org/info/rfc2818>

[RFC3280] Housley, R., Polk, W., Ford, W., and Solo, D., "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile", RFC 3280, April 2002, <http://www.rfc-editor.org/info/rfc3280>

[RFC3339] Klyne, G. and Newman, C., "Date and Time on the Internet: Timestamps", RFC 3339, July 2002, <https://www.rfc-editor.org/info/rfc3339>

[RFC4158] Cooper, M., Dzambasow, Y., Hesse, P., et al., "Internet X.509 Public Key Infrastructure: Certification Path Building", RFC 4158, September 2005, <https://www.rfc-editor.org/info/rfc4158>

[RFC4559] Jaganathan, K., Zhu, L., and Brezak, J., "SPNEGO-based Kerberos and NTLM HTTP Authentication in Microsoft Windows", RFC 4559, June 2006, <https://www.rfc-editor.org/info/rfc4559>

[RFC4648] Josefsson, S., "The Base16, Base32, and Base64 Data Encodings", RFC 4648, October 2006, <https://www.rfc-editor.org/info/rfc4648>

[RFC7519] Internet Engineering Task Force, "JSON Web Token (JWT)", <https://www.rfc-editor.org/info/rfc7519>

[RFC793] Postel, J., Ed., "Transmission Control Protocol: DARPA Internet Program Protocol Specification", RFC 793, September 1981, <https://www.rfc-editor.org/info/rfc793>

[SAMLCore2] Cantor, S., Kemp, J., Philpott, R., and Maler, E., Eds., "Assertions and Protocol for the OASIS Security Assertion Markup Language (SAML) V2.0", March 2005, <http://docs.oasis-open.org/security/saml/v2.0/saml-core-2.0-os.pdf>

[WSFederation1.2] Kaler, C., McIntosh, M., "Web Services Federation Language (WS-Federation)", Version 1.2, May 2009, <http://docs.oasis-open.org/wsfed/federation/v1.2/os/ws-federation-1.2-spec-os.html>

1.2.2 Informative References

[MS-ADFSOD] Microsoft Corporation, "[Active Directory Federation Services \(AD FS\) Protocols Overview](#)".

[MSFT-ADFS-DeepDive] Gregory, David, "ADFS Deep-Dive: Comparing WS-Fed, SAML, and OAuth", <https://techcommunity.microsoft.com/t5/core-infrastructure-and-security/adfs-deep-dive-comparing-ws-fed-saml-and-oauth/ba-p/257584>

[MSFT-ADFSOV2] Microsoft Corporation, "Active Directory Federation Services", <https://learn.microsoft.com/en-us/windows-server/identity/active-directory-federation-services>

[RFC6101] Freier, A., Karlton, P., and Kocher, P., "The Secure Sockets Layer (SSL) Protocol Version 3.0", RFC 6101, August 2011, <http://www.rfc-editor.org/rfc/rfc6101.txt>

[RFC6749] Hardt, D., Ed., "The OAuth 2.0 Authorization Framework", RFC 6749, October 2012, <https://www.rfc-editor.org/info/rfc6749>

[WSFederation] Kaler, C., Nadalin, A., Bajaj, S., et al., "Web Services Federation Language (WS-Federation)", Version 1.1, December 2006, <http://specs.xmlsoap.org/ws/2006/12/federation/ws-federation.pdf>

1.3 Overview

The **Active Directory Federation Services (AD FS)** Proxy and Web Application Proxy Integration Protocol comprise the **Active Directory Federation Services and Proxy system** that provides services for authentication, authorization, and access to application services located inside the boundaries of the corporate network for clients that are located outside that boundary.

AD FS is located inside the boundaries of the corporate network. It can run on one server, or multiple servers known as a **farm configuration**. AD FS is a collection of authentication and authorization services exposed to clients over the HTTP protocol [\[RFC2616\]](#). AD FS implements a set of application authentication protocols including WS-Federation [\[WSFederation\]](#), SAML-P [\[SAMLCore2\]](#), and OAuth [\[RFC6749\]](#).

The Proxy is a service located at the edge of the corporate network. It provides **proxy** services for clients requesting access to application services inside the corporate network and orchestrates access traffic to these services. The Proxy directs all authentication traffic to the AD FS in the **internal network** and provisions for certificate-based authentication.

The Proxy publishes application services that are located inside the boundaries of the corporate network and makes them available for access to clients that are outside. It gates the access to the network and provides **pre-authentication** by orchestrating the authentication to the edge through AD FS before allowing the access to the application service. AD FS defines and implements a protocol that the Proxy supports and that allows the Proxy to orchestrate access to the network by authenticating requests to the edge.

The following diagram illustrates the various components of the system.

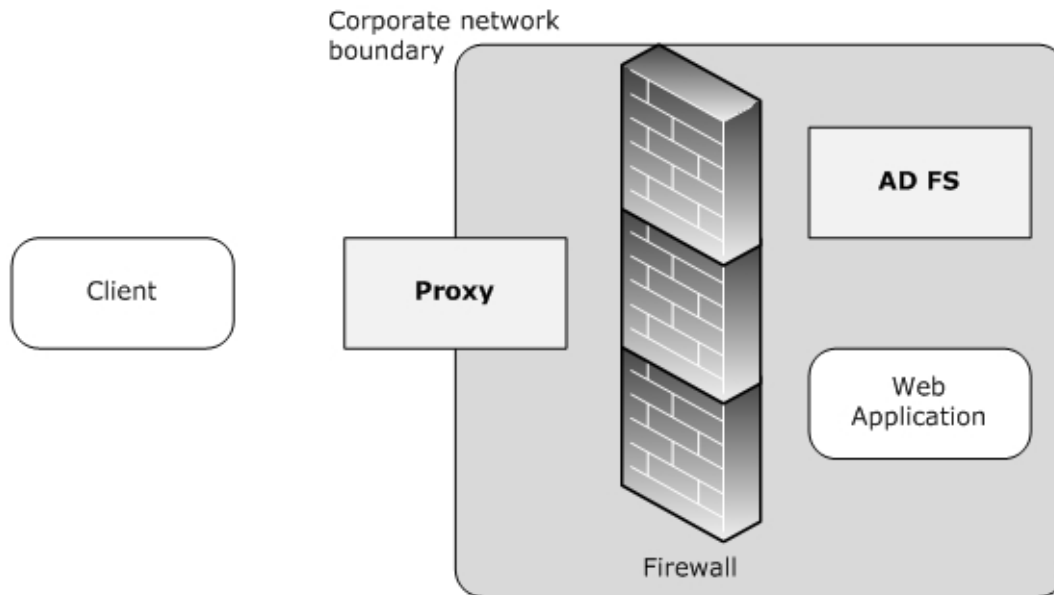


Figure 1: ADFSPIP System components

The following components are part of the Active Directory Federation Services and Proxy system:

- **AD FS:** A federation services provider. In this specification this component will be referred to as the server.
- **Proxy:** Both an authentication and an application proxy. In this specification this component will be referred to as the client.

The following components interact with the Active Directory Federation Services and Proxy system:

- **Client:** These components refer to the type of client (for example, browser or rich client) in addition to the identity of the user and the device that is accessing a particular application service.
- **Firewall:** A component that filters traffic flowing between the **perimeter network** and the internal network. In the system described, web traffic is allowed between the Proxy and the AD FS and between the Proxy and the web application.
- **Web Application:** Any web service or application to which a client connects and that typically requires authentication for the user in the client.

This specification describes the distinct areas of interaction between the Proxy and the AD FS. A more complete understanding of AD FS and the proxy system can be gained from the following resources: [\[MS-ADFSOD\]](#), [\[MSFT-ADFSOV2\]](#) with its subsections, and [\[MSFT-ADFS-DeepDive\]](#).

1.4 Relationship to Other Protocols

The following figure illustrates the relationship of this protocol to other protocols.

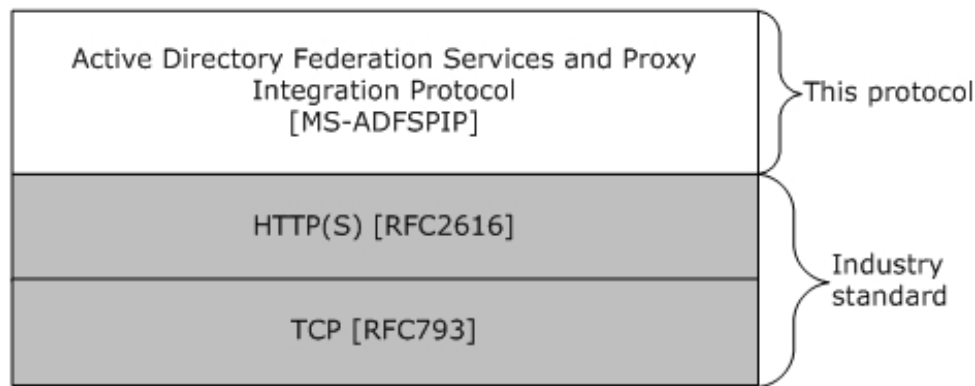


Figure 2: Protocols related to the Active Directory Federation Services and Proxy Integration Protocol

This protocol uses TCP [\[RFC793\]](#) as its transport.

Where specified, this protocol uses base64url encoding ([\[RFC4648\]](#) section 5).

1.5 Prerequisites/Preconditions

No prerequisites or preconditions.

1.6 Applicability Statement

The protocols in the **Active Directory Federation Services and Proxy system** are applicable to any situation in which the following are important:

1. A **proxy** for **AD FS**.
2. Publishing of web applications or services behind-the-firewall to the Internet.
3. **Pre-authentication** of clients accessing web applications or services behind a firewall.

1.7 Versioning and Capability Negotiation

This protocol does not provide any mechanism for capability negotiation.

1.8 Vendor-Extensible Fields

This protocol does not provide any vendor-extensible fields.

1.9 Standards Assignments

This protocol has not been assigned any standard parameters.

2 Messages

2.1 Transport

The protocol is transported by **HTTP/HTTPS** [\[RFC2616\]](#) [\[RFC2818\]](#). The protocol requires HTTP/HTTPS ports as specified in section [2.2.2.4](#), attributes "HttpPort", "HttpsPort" and "HttpsPortForUserTlsAuth", obtained during Proxy (that is, the **Web Application Proxy**) server registration (section [3.4.5.1](#)).

2.2 Common Data Types

This section defines the set of resource types that are consumed or produced by this protocol. Common element definitions are included in this section.

2.2.1 HTTP Headers

The following table summarizes the set of HTTP Headers defined by this specification.

Header	Description
X-MS-Endpoint-Absolute-Path	section 2.2.1.3
X-MS-Forwarded-Client-IP	section 2.2.1.2
X-MS-Proxy	section 2.2.1.1
X-MS-Target-Role	section 2.2.1.4
X-MS-ADFS-Proxy-Client-IP	section 2.2.1.5
X-MS-ProxyAuth-Token	section 2.2.1.6

2.2.1.1 X-MS-Proxy

This header MUST contain the value of the server name of the **proxy**. This header is included when the proxy is processing client incoming requests as described in the runtime behaviors for the **AD FS** proxy server details in section [3.6](#).

```
String = *(%x20-7E)
X-MS-Proxy = String
```

2.2.1.2 X-MS-Forwarded-Client-IP

This header MUST contain the value of the IP address of the client sending the request. This header MUST be included when the **proxy** is processing incoming requests from clients trying to access the server.

```
String = *(%x20-7E)
X-MS-Forwarded-Client-IP = String
```


2.2.1.3 X-MS-Endpoint-Absolute-Path

This header MUST contain the full URL of the incoming request. This header MUST be included when the **proxy** is processing incoming requests from clients trying to access the server.

```
String = *(%x20-7E)
X-MS-Endpoint-Absolute-Path = String
```

2.2.1.4 X-MS-Target-Role

This header MUST contain the value "PrimaryComputer" to specify that a given HTTP GET request MUST perform the fetch on a server that has both read and write capabilities on the data.

```
String = *(%x20-7E)
X-MS-Target-Role = String
```

2.2.1.5 X-MS-ADFS-Proxy-Client-IP

The value of this header MUST be set to the IP address of the client's TCP/IP connection to the **proxy**. This header SHOULD be included when the proxy is processing incoming requests from clients that are trying to access the server. [<1>](#1)

```
String = *(%x20-7E)
X-MS-ADFS-Proxy-Client-IP = String
```

2.2.1.6 X-MS-ProxyAuth-Token

The value of this header MUST be set to a serialized **JSON Web Token (JWT)** bearer request containing details of the token binding information. This header SHOULD be included when the proxy is processing incoming requests from clients that are trying to access the server, and which contain token binding information in the form of the Sec-Token-Binding header defined in [\[IETFDRAFT-TOKBIND-H\]](#) section 2. [<2>](#2)

```
String = *(%x20-7E)
X-MS-ProxyAuth-Token = String
```

This field must be a serialized JWT, as defined by [\[RFC7519\]](#).

The signing key must be a trust certificate. The x5t header on the JWT must be the byte data of the trust certificate.

```
{ "SerializedTrustCertificate" : "<certificate>" }
```

The JWT must use the RSA-SHA256 signature algorithm.

The JWT must contain the following two claims, matching the headers specified in [\[IETFDRAFT-TOKBIND-T\]](#) section 2.3:

```
"Sec-Provided-Token-Binding-ID": <provided binding ID from Sec-Token-Binding header on
original request>
```

"Sec-Referred-Token-Binding-ID": <referred binding ID from Sec-Token-Binding header on original request>

2.2.2 Complex Types

The following are the defined types used by the protocol details.

2.2.2.1 Proxy Trust

This is a **JavaScript Object Notation (JSON)** object containing a trust certificate. The format of the object is as follows:

```
{ "SerializedTrustCertificate" : "<certificate>" }
```

certificate: Base64 string encoded ([\[RFC4648\]](#) section 4) X509 certificate [\[RFC4158\]](#).

2.2.2.2 Proxy Trust Renewal

This is a JSON object containing a new trust certificate. The format of the object is as follows:

```
{ "SerializedReplacementCertificate" : "<certificate>" }
```

certificate: Base64 string encoded ([\[RFC4648\]](#) section 4) X509 certificate [\[RFC4158\]](#).

2.2.2.3 Proxy Relying Party Trust

This is a JSON object containing the identifier of the web application for the **proxy**. The format of the object is as follows:

```
{ "Identifier" : "<web-application-for-client-id>" }
```

web-application-for-client-id: URI of the web application representing the client. The server will issue **tokens** with this value as the audience as described in section [3.13](#).

2.2.2.4 Configuration

This is a JSON object containing information about the **AD FS** service. The format of the object is as follows:

```
{
  "ServiceConfiguration" :
  {
    "ServiceHostName" : "<service-host-name>",
    "HttpPort" : <http-port-number>,
    "HttpsPort" : <https-port-number>,
    "HttpsPortForUserTlsAuth" : <user-TLS-port-number>,
    "DeviceCertificateIssuers" : [ "<device-certificate-issuer>", * ],
    "ProxyTrustCertificateLifetime" : <trust-renewal-interval>,
    "DiscoveredUpnSuffixes" : [ "<upn-suffix>", * ],
    "CustomUpnSuffixes" : [ "<upn-suffix>", * ],
    "ServiceHostNameForUserTlsAuth" : "<service-host-name-for-user-tls-auth>"
  },
  "EndpointConfiguration" : {
```

```

    "Endpoints" : [
      {
        "Path" : "<endpoint-uri>",
        "PortType" : "<port-type>",
        "AuthenticationSchemes" : "<credential-collection-scheme>",
        "ClientCertificateQueryMode" : "<tls-query-behavior>",
        "CertificateValidation" : "<certificate-validation>",
        "SupportsNtlm" : "<support-ntlm>",
        "ServicePath" : "<service-endpoint-uri>",
        "ServicePortType" : "<service-port-type>"
      }, *
    ]
  },
  "FarmBehavior" : "<farm-behavior-version-number>",
  "IgnoreTokenBinding" : "<ignore-token-binding>"
  "UpdatedFarmBehaviorLevel" : "<updated-farm-behavior-level>"
}

```

service-host-name: Host name of the AD FS service.

service-host-name-for-user-tls-auth: (Optional) Alternate hostname of the AD FS service that implements the endpoint used to authenticate the user using **Transport Layer Security (TLS)** authentication. [<3>](#3)

http-port-number: Port number for endpoints listening on HTTP.

https-port-number: Port number for endpoints listening on HTTPS.

user-tls-port-number: Port number for user TLS authentication endpoints.

device-certificate-issuer: Base64 string encoded ([\[RFC4648\]](#) section 4) X509 certificate [\[RFC4158\]](#).

trust-renewal-interval: Hint for **proxy** certificate lifetime.

upn-suffix: Possible User Principal Name (UPN) suffixes for principals that can be preauthorized.

endpoint-uri: URI of endpoint.

port-type: Port Type (section [2.2.2.12](#)) for endpoint.

credential-collection-scheme: Credential Collection Scheme (section [2.2.2.13](#)) for endpoint.

tls-query-behavior: TLS Query Behavior (section [2.2.2.14](#)) for endpoint.

certificate-validation: Certificate Validation (section [2.2.2.15](#)) for endpoint.

support-ntlm: Boolean value that indicates whether the client supports NTLM authentication for SPNEGO-based HTTP authentication [\[RFC4559\]](#).

service-endpoint-uri: URI of endpoint on server. This URI is relative to service-host-name.

service-port-type: Port Type (section 2.2.2.12) for corresponding endpoint on server.

farm-behavior-version-number: (Optional) The following table shows the values of **farm-behavior-version-number** corresponding to the **ad_fs_behavior_level** setting ([\[MS-OAPX\]](#) section 3.2.1.1) on the server. [<4>](#4)

ad_fs_behavior_level	farm-behavior-version-number
AD_FS_BEHAVIOR_LEVEL_1	"6.3"
AD_FS_BEHAVIOR_LEVEL_2	"10.0"
AD_FS_BEHAVIOR_LEVEL_3	"10.0"
AD_FS_BEHAVIOR_LEVEL_4	"10.0"

If this value is not specified, the value of "6.3" is assumed.

ignore-token-binding: (Optional) A Boolean attribute on the server indicating that token binding information [\[IETF DRAFT-TOKBND\]](#) is not to be retrieved from http.sys for a request and is to be ignored in any existing tokens. The default is true. [<5>](#)

updated-farm-behavior-level: (Optional) An integer attribute on the server that specifies the forward-compatible **AD FS farm** behavior level. Note that this is different from the **farm-behavior-version-number** field. This value corresponds directly to the **ad_fs_behavior_level** setting on the server ([MS-OAPX] section 3.2.1.1). [<6>](#)

2.2.2.5 Relying Party Trust List

This is a JSON array of objects containing web application information. The format of the objects is as follows:

```
[ {
  "objectIdentifier" : "<object-identifier>",
  "name" : "<web-application-name>",
  "publishedThroughProxy" : <is-web-application-published>,
  "nonClaimsAware" : <is-a-non-claims-aware-web-application>,
  "enabled" : <is-web-application-enabled>
}, + ]
```

object-identifier: The immutable object identifier for the web application on the server.

web-application-name: The name of the web application on the server, unique across web applications.

is-web-application-published: Boolean user configuration declaring this web application as being accessible from outside the **internal network** through a client.

is-a-non-claims-aware-web-application: Boolean value specifying if the web application is a **non-claims-aware** web application.

enabled: Boolean value specifying if the web application is enabled at the server.

2.2.2.6 Relying Party Trust

This is a JSON object containing detailed web application information. The format of the object is as follows:

```
{
```

```

    "objectIdentifier" : "<object-identifier>",
    "name" : "<web-application-name>",
    "publishedThroughProxy" : <is-web-application-published>,
    "nonClaimsAware" : <is-a-non-claims-aware-web-application>,
    "enabled" : <is-web-application-enabled>,
    "identifiers" : [ <web-application-identifier>, * ],
    "proxyTrustedEndpoints" : [ <web-application-at-proxy-endpoint-url>, * ],
    "proxyEndpointMappings" :
      [ { "Key" = "<internal-url>", "Value" = "<external-url>" }, * ]
  }

```

object-identifier: The unique object identifier for the web application.

web-application-name: The name of the web application on the server, unique across web applications.

is-web-application-published: Boolean user configuration declaring this web application as accessible from outside the **internal network** through a client. This value MUST correspond to the value of (proxyTrustedEndpoints.Count > 0).

is-a-non-claims-aware-web-application: Boolean value specifying if the web application is a **non-claims-aware** web application.

is-web-application-enabled: Boolean value specifying if the web application is enabled at the server.

web-application-identifier: An identifier of the web application on the server.

web-application-at-proxy-endpoint-url: A URL representing an endpoint on the client for the web application where the server will issue **tokens** to.

internal-url: The internal URL corresponding to the internal-to-external mapping.

external-url: The external URL corresponding to the internal-to-external mapping.

2.2.2.7 Relying Party Trust Publishing Settings

This is a JSON object containing web application publishing information. The format of the object is as follows:

```

{
  "externalUrl" : "<external-url>",
  "internalUrl" : "<internal-url>",
  "proxyTrustedEndpoint" : "<web-application-at-proxy-url>"
}

```

external-url: The external URL to be associated with the web application external-to-internal mappings (section [2.2.2.6](#)).

internal-url: The internal URL to be associated with the web application external-to-internal mappings (section [2.2.2.6](#)).

web-application-at-proxy-url: The URL of the endpoint in the client where the server will issue **tokens** to.

2.2.2.8 Store Entry List

This is a JSON array of store entry objects, which are defined in section [2.2.2.9](#).

2.2.2.9 Store Entry

This is a JSON object containing store entry information. The format of the object is as follows:

```
{
  "key" : "<entry-key>",
  "version" : <entry-version>,
  "value" : "<entry-value>"
}
```

entry-key: A string that contains the key of the data value for the store entry.

entry-version: A value that specifies the version of the key/value pair for the store entry.

entry-value: The value of the data-blob corresponding to the given key for the store entry.

2.2.2.10 Store Entry Key and Value

This is a JSON object containing the value of a store entry. The format of the object is as follows:

```
{
  "key" : "<entry-key>",
  "value" : "<entry-value>"
}
```

entry-key: A string containing the key of the data value for the store entry.

entry-value: The value of the data-blob corresponding to the given key for the store entry.

2.2.2.11 Serialized Request with Certificate

This is a JSON object containing a serialized HTTP request that is intended for the target service, plus a serialized client certificate and its usage. The format of the object is as follows:

```
{
  "Request" :
  {
    "AcceptTypes" : [ "<accept-type>", * ],
    "Content" : [ <byte>, * ],
    "ContentEncoding" : "<content-encoding>",
    "ContentLength" : <content-length>,
    "ContentType" : "<content-type>",
    "Cookies" :
    [ {
      "Name" : "<cookie-name>",
      "Value" : "<cookie-value>",
      "Path" : "<cookie-path>",
      "Domain" : "<cookie-domain>",
      "Expires" : <cookie-expires>,
      "Version" : <cookie-version>,
    }, * ],
    "Headers" :
    [ { "Name" : "<header-name>", "Value" : "<header-value>" }, * ],
    "HttpMethod" : "<http-method>",
    "RequestUri" : "<request-uri>",
    "QueryString" : [ { "Name" : "<query-param>", "Value" : "<query-value>" }, * ],
    "UserAgent" : "<user-agent>",
    "UserHostAddress" : "<user-host-address>",
    "UserHostName" : "<user-host-name>",
    "UserLanguages" : [ "<user-language>", * ]
  },
}
```

```

    "SerializedClientCertificate" : "<serialized-client-certificate>",
    "CertificateUsage" : "<certificate-usage>",
    "ErrorType" : "<Error-Type>",
    "ErrorCode" : "<Error-Code>"
}

```

accept-type: A string that represents a MIME accept type supported by the client. This corresponds to a value of the Accept header of the request.

byte: An 8-bit integer in decimal form.

content-encoding: Character set of the entity-body of the request.

content-length: Length in bytes of content sent in the request.

content-type: MIME content type of the request.

cookie-name: Name of the cookie.

cookie-value: Value of the cookie.

cookie-path: Virtual path transmitted with the cookie.

cookie-domain: Domain associated with the cookie.

cookie-expires: Expiration date and time of the cookie.

cookie-version: Version of the cookie.

header-name: Name of header.

header-value: Value of header.

http-method: HTTP data transfer method of the request, for example GET, POST, HEAD.

request-uri: URI of the request.

query-param: Name of the query parameter.

query-value: Value of the query parameter.

user-agent: User agent presented in the request.

user-host-address: IP address and port number to which the request was directed.

user-host-name: DNS name and port number (if provided) specified in the request.

user-language: Natural language preferred for the response.

serialized-client-certificate: Client certificate obtained from **TLS** handshake base64 string encoded.

certificate-usage: Certificate Type (section [2.2.2.16](#)) for certificate.

Error-Type: Error Type (section 2.2.2.17).[<7>](#)

Error-Code: Error code, as an integer.[<8>](#)

2.2.2.12 Port Type

This is an enumeration with the following integer values:

```
{
  0
  1
  2
}
```

0: The value for the HTTP port, "HttpPort".

1: The value for the HTTPS port, "HttpsPort".

2: The value for the HTTPS port for user **TLS** authentication, "HttpsPortForUserTlsAuth".

2.2.2.13 Credential Collection Scheme

This is an enumeration with the following integer values indicating the type of credential to collect from the client:

```
{
  8
  32768
}
```

8: Basic authentication credentials.

32768: Anonymous authentication.

2.2.2.14 TLS Query Behavior

This is an enumeration with the following integer values:

```
{
  0
  1
  2
}
```

0: The value for "None".

1: The value for "QueryAndAccept".

2: The value for "QueryAndRequire".

2.2.2.15 Certificate Validation

This is an enumeration with the following integer values:

```
{
  0
  1
  2
}
```

0: The value for "None".

1: The value for "Ssl".

2: The value for "IssuedByDrs".

2.2.2.16 Certificate Type

This is an enumeration with the following integer values:

```
{
  1
  2
}
```

1: The value for the user certificate, "User".

2: The value for the device certificate, "Device".

2.2.2.17 Error Type

This is an enumeration with the following integer values:

```
{
  0
  1
}
```

0: The value for "None".

1: The value for "Certificate".

2.2.2.18 Proxy Token

This is a JSON object representing the **token** issued to the client. The format of the object is defined in [\[IETF DRAFT-JWS\]](#) and is as follows:

```
{
  "ver" : "<version>",
  "aud" : "<audience>",
  "iat" : <issued-at>,
  "exp" : <expire>,
  "iss" : "<issuer>",
  "relyingpartytrustid" : "<rp-trust-id>",
  "deviceregid" : "<device-registration-id>",
  "authinstant" : <auth-instant>,
  "authmethod" : "<auth-method>",
  "upn" : "<upn>"
}
```

version: Token version with a value of 1.0.

audience: Audience for this token. The **proxy** SHOULD verify that this value matches the value for [Client State].ProxyRelyingPartyTrustIdentifier.

issued-at: Issued at date and time. The proxy SHOULD verify that this value corresponds to a time in the past (before the current time). This is a JSON numeric value representing the number of seconds from 1970-01-01T0:0:0Z **Coordinated Universal Time (UTC)** until the specified UTC date/time. See [\[RFC3339\]](#) for details regarding date/times in general and UTC in particular.

expire: Expiration time of token. The proxy SHOULD verify that this value corresponds to a time in the future (after the current time). This is a JSON numeric value representing the number of seconds from 1970-01-01T0:0:0Z UTC until the specified UTC date/time. See [RFC3339] for details regarding date/times in general and UTC in particular.

issuer: Trusted issuer for this token. The proxy SHOULD verify that this value corresponds to the issuer URI that is published by the server issuing this token through its Federation Metadata [\[WSFederation1.2\]](#).

rp-trust-id: GUID representing application being accessed. The proxy MAY use this value to correlate requests and tokens when listening to multiple requests.

device-registration-id: Identity of the device attempting the access in the form of its certificate thumbprint. The proxy MAY use this value to correlate the client of the request with the client of the token.

auth-instant: Time of authentication. The proxy SHOULD verify that this value corresponds to an earlier time than the issued-at value.

auth-method: Authentication method. The proxy MAY use this value to perform richer authorization of access.

upn: User Principal Name (UPN) of user attempting the access.

2.2.2.19 Combined Token

This is a JSON object containing an access **token** for the client and an access token for the web application. The format of the object is as follows:

```
{
  "proxy_token" : "<proxy-token>",
  "access token" : "<access-token>"
}
```

proxy-token: [Proxy Token] (section [2.2.2.18](#)).

access-token: Token issued by the server to the web application.

2.2.2.20 Proxy Token Wrapper

This is a JSON object containing a **proxy token** as a value on the object. The format of the object is as follows:

```
{
  "authToken" : "<proxy-token>"
}
```

proxy-token: A base64 string encoded ([\[RFC4648\]](#) section 4) [Proxy Token] (section [2.2.2.18](#)).

2.2.2.21 Authentication Request

This is a JSON object containing an authentication request. The format of the object is as follows:

```
{
  "appRealm" : "<web-application-id>",
  "realm" : "<web-application-for-client-id>",
  "username" : "<username>",
}
```

```

    "password" : "<password>",
    "deviceCertificate" : "<device-certificate>",
    "userCertificate" : "<user-certificate>",
    "httpHeaders" :
      [ { "Key" : "<header-name>", "Value" : "<header-value>" }, * ]
  }

```

web-application-id: The identifier of the target relying party.

web-application-for-client-id: The identifier of the WAP relying party.

username: The username of the target user.

password: The password of the target user in a base-64-url encoded string.

device-certificate: The certificate used for the device registration in byte[] serialized as the base-64 encoded string.

user-certificate: The certificate to be used to authenticate the user in byte[] serialized as the base-64 encoded string.

header-name: (string) The HTTP header name.

header-value: (string) The value of the corresponding HTTP header.

2.2.2.22 Error Response

This is a JSON object containing exception or error data that the **proxy** receives from the server to build a response to the client. The format of the object is as follows:

```

{
  "id" : <error-id>,
  "message" : "<message>",
  "type" : "<type>",
}

```

error-id: (DWORD) The identifier of the error encountered. This parameter is not required and can be empty. The following error identifiers can be returned:

- 401 (Unauthorized) – The username, password combination or the user certificate provided is not valid.
- 403 (Forbidden) – The given user is not authorized to access the given relying party. The authorization rules of either the target relying party or the WAP relying party need to be modified.
- 404 (Not Found) – The target relying party or the WAP relying party is not found.
- 412 (Precondition Failed) - If the relying party rules require additional authentication. The additional rules of either the target relying party of the WAP relying party need to be modified.

message: (string) The message corresponding to the error in the user locale of the STS.

type: (string) Additional debug information.

3 Protocol Details

3.1 Common Details

3.1.1 Abstract Data Model

This section describes a conceptual model of possible data organization that an implementation of the client and server maintain to participate in this protocol. The described organization is provided to facilitate the explanation of how the protocol behaves. This document does not mandate that implementations adhere to this model as long as their external behavior is consistent with that described in this document.

3.1.1.1 Server State

The following represents the data structure the server **MUST** hold in order to satisfy these protocol requirements^{<9>}:

```
{
  "ProxyTrustedCertificates" : [ "<certificate-identifier>", * ],
  "ProxyRelyingPartyTrust" : "<web-application-for-proxy>",
  "Configuration" : "<configuration>",
  "RelyingPartyTrusts" : [ "<web-application>", * ],
  "ProxyStore" : [ "StoreEntry" : "<store-entry>", * ]
}
```

certificate-identifier: Data that **MUST** be used to validate the certificate when presented again.

web-application-for-proxy: Proxy Relying Party Trust (section [2.2.2.3](#)) representing the web application for the client in the server.

configuration: Configuration (section [2.2.2.4](#)) representing service and endpoint configuration.

web-application: Relying Party Trust (section [2.2.2.6](#)) representing an available web application in the server.

store-entry: Store Entry (section [2.2.2.9](#)) containing the triplet of key-version-value of data used by the client for its own consumption.

3.1.1.2 Client State

The following represents the data structure the **proxy** service **MUST** hold in order to satisfy these protocol requirements:

```
{
  "TrustCertificate" : "<certificate-with-private-key>",
  "ProxyRelyingPartyTrustIdentifier" : "<web-application-for-client-id>",
  "Configuration" : "<configuration>",
  "RelyingPartyTrusts" : [ "<web-application>", * ]
}
```

certificate-with-private-key: Points to a certificate. The proxy service **MUST** have a private key for the certificate.

web-application-for-client-id: Identifier of the web application representing the client on the server. This identifier **MUST** be used by the client when referring to itself on requests to the server.

configuration: Configuration (section [2.2.2.4](#)) obtained from the server.

web-application: Relying Party Trust State (section [3.1.1.3](#)) containing the configuration for a web application on the server.

3.1.1.3 Relying Party Trust State

The following represents the data structure the client MUST hold in order to satisfy these protocol requirements:

```
{
  "RelyingPartyTrust" : "<web-application>",
  "RedirectBasedPreauth" : <redirect-based-preauth>
}
```

web-application: Relying Party Trust (section [2.2.2.6](#)) representing the web application that the server can issue **tokens** for.

redirect-based-preauth: Boolean denoting that access from outside the network needs **pre-authentication** based on HTTP redirects.

3.1.2 Timers

None.

3.1.3 Initialization

None.

3.1.4 Higher-Layer Triggered Events

None.

3.1.5 Message Processing Events and Sequencing Rules

None.

3.1.6 Timer Events

None.

3.1.7 Other Local Events

None.

3.2 Proxy Registration Server Details

3.2.1 Abstract Data Model

None.

3.2.2 Timers

None.

3.2.3 Initialization

None.

3.2.4 Higher-Layer Triggered Events

None.

3.2.5 Message Processing Events and Sequencing Rules

For the system to function properly, the client and the server **MUST** mutually authenticate each other using client TLS authentication [\[RFC2246\]](#). For this, the client **MUST** have the appropriate local configuration to evaluate the trustworthiness of the server TLS certificate and **MUST** have a client TLS certificate for authenticating itself to the server.

The following resources are required to create and maintain a proper trust configuration between the client and the server.

Resource	Description
Proxy/EstablishTrust	Resource used to establish a trust with the server.
Proxy/RenewTrust	Resource used to renew the trust with the server.

The responses to all the operations can result in the following status codes.

Status code	Description
200	The operation has succeeded.
400	The request is not valid.
401	Unauthorized for specified user credentials or for client TLS certificate.
404	The object does not exist.
405	Invalid verb used in request (GET, DELETE, POST, PUT).
409	The object already exists.
500	Version is not specified where required or any other internal error.
501	Version specified (api-version) is invalid (only valid value is 1).

If the operation authenticates using Integrated Windows authentication [\[RFC2478\]](#), the server **MUST** validate that the authenticated principal is authorized to do the corresponding operation on the server.

3.2.5.1 Proxy/EstablishTrust

The client **MUST** first establish a trust with the server in order to act as a Proxy on the system.

3.2.5.1.1 POST

This operation creates a trust based on a Proxy Trust (section [2.2.2.1](#)).

The operation is transported by a HTTP **POST** and can be invoked through the following URIs:

```
ads/proxy/EstablishTrust
ads/proxy/PrimaryWriter/EstablishTrust
```

If the operation is invoked through `ads/proxy/EstablishTrust`, the request **MUST** authenticate using HTTP Basic authentication [\[RFC2617\]](#).

If the operation is invoked through `ads/proxy/PrimaryWriter/EstablishTrust`, the request **MUST** authenticate using Integrated Windows authentication [\[RFC2478\]](#).

The response message for this operation can result in the following status codes.

Status code
200
400
401
500

3.2.5.1.1.1 Request Body

The request body **MUST** be a Proxy Trust (section [2.2.2.1](#)).

3.2.5.1.1.2 Response Body

No response body is returned.

3.2.5.1.1.3 Processing Details

If the operation authenticates using HTTP Basic authentication [\[RFC2617\]](#), the server **MUST** validate that the authenticated principal is authorized to function as a **proxy**.

The server **MUST** validate that the [Proxy Trust].SerializedTrustCertificate has an **extended key usage (EKU)** for client authentication (1.3.6.1.5.5.7.3.2) ([\[RFC3280\]](#) section 4.2.1.13) and is within the validity period ([\[RFC1422\]](#) section 3.3). If validation fails, the server **MUST** return a HTTP error code of 400.

On successful authentication and authorization, the server **MUST** add [Proxy Trust].SerializedTrustCertificate to [Server State].ProxyTrustedCertificates for future validations.

3.2.5.2 Proxy/RenewTrust

The client **MUST** ensure that the trust with the server remains valid by renewing the trust certificate with the server.

3.2.5.2.1 POST

This operation renews a trust based on a Proxy Trust Renewal (section [2.2.2.2](#)).

The operation is transported by a HTTP **POST** and can be invoked through the following URIs:

```
ads/proxy/RenewTrust
ads/proxy/PrimaryWriter/RenewTrust
```

If the operation is invoked through `adfs/proxy/RenewTrust`, the request MUST authenticate using client TLS authentication [\[RFC2246\]](#). The server MUST validate that the certificate presented by the client during client TLS authentication [\[RFC2246\]](#) can be validated by one of the values of `[Server State].ProxyTrustedCertificates`. If the certificate cannot be validated, the server MUST return a HTTP error code of 400.

If the operation is invoked through `adfs/proxy/PrimaryWriter/RenewTrust`, the request MUST authenticate using Integrated Windows authentication [\[RFC2478\]](#).

The response message for this operation can result in the following status codes.

Status code
200
400
401
500

3.2.5.2.1.1 Request Body

The request body MUST be Proxy Trust Renewal (section [2.2.2.2](#)).

3.2.5.2.1.2 Response Body

No response body is returned.

3.2.5.2.1.3 Processing Details

The server MUST validate that the `[Proxy Trust].SerializedReplacementCertificate` has an **extended key usage (EKU)** for client authentication (1.3.6.1.5.5.7.3.2) ([\[RFC3280\]](#) section 4.2.1.13) and is within the validity period ([\[RFC1422\]](#) section 3.3). If validation fails, the server MUST return a HTTP error code of 400.

The server MUST add `[Proxy Trust].SerializedReplacementCertificate` to `[Server State].ProxyTrustedCertificates` for future validations.

3.2.5.3 Proxy/WebApplicationProxy/Trust

The client MUST register with the server as a **token** recipient with the server before it can function as the Proxy on the system.

3.2.5.3.1 GET

This operation returns a Proxy Relying Party Trust (section [2.2.2.3](#)) corresponding to the web application for the client in the server.

The operation is transported by a HTTP **GET** and can be invoked through the following URI:

```
adfs/proxy/WebApplicationProxy/trust?api-version=1
```

The request MUST authenticate using client TLS authentication [\[RFC2246\]](#). The server MUST validate that the certificate presented by the client during client TLS authentication [\[RFC2246\]](#) can be validated by one of the values of `[Server State].ProxyTrustedCertificates`. If the certificate cannot be validated, the server MUST return an HTTP error code of 401.

The response message for this operation can result in the following status codes.

Status code
200
400
401
404
500
501

3.2.5.3.1.1 Request Body

The server MUST ignore any request body.

3.2.5.3.1.2 Response Body

The response body MUST be a Proxy Relying Party Trust (section [2.2.2.3](#)).

3.2.5.3.1.3 Processing Details

On successful authentication the server MUST return [Server State].ProxyRelyingPartyTrust (section [3.1.1.1](#)).

3.2.5.3.2 POST

This operation creates the **proxy** relying party trust based on a Proxy Relying Party Trust (section [2.2.2.3](#)).

The operation is transported by a HTTP **POST** and can be invoked through the following URIs:

```
adfs/proxy/WebApplicationProxy/trust?api-version=1
adfs/proxy/WebApplicationProxy/PrimaryWriter/trust?api-version=1
```

If the operation is invoked through `adfs/proxy/WebApplicationProxy/trust?api-version=1`, the request MUST authenticate using client TLS authentication [\[RFC2246\]](#).

If the operation is invoked through `adfs/proxy/WebApplicationProxy/PrimaryWriter/trust?api-version=1`, the request MUST authenticate using Integrated Windows authentication [\[RFC2478\]](#).

The response message for this operation can result in the following status codes.

Status code
200
400
401
409

Status code
500
501

3.2.5.3.2.1 Request Body

The request body MUST be a Proxy Relying Party Trust (section [2.2.2.3](#)).

3.2.5.3.2.2 Response Body

No response body is returned.

3.2.5.3.2.3 Processing Details

On successful authentication the server MUST verify that [Server State].ProxyRelyingPartyTrust is not set.

If it is set, the server MUST return a HTTP error code of 409.

If it is not set, the server MUST create the relying party trust for the **proxy** with an identifier of the received [Proxy Relying Party Trust].Identifier and set the [Server State].ProxyRelyingPartyTrust to the value of the received Proxy Relying Party Trust (section [2.2.2.3](#)).

3.2.5.3.3 DELETE

This operation removes the **proxy** relying party trust.

The operation is transported by a HTTP **DELETE** and can be invoked through the following URIs:

```

ads/proxy/WebApplicationProxy/trust?api-version=1
ads/proxy/WebApplicationProxy/PrimaryWriter/trust?api-version=1

```

If the operation is invoked through `ads/proxy/WebApplicationProxy/trust?api-version=1`, the request MUST authenticate using client TLS authentication [\[RFC2246\]](#).

If the operation is invoked through `ads/proxy/WebApplicationProxy/PrimaryWriter/trust?api-version=1`, the request MUST authenticate using Integrated Windows authentication [\[RFC2478\]](#).

The response message for this operation can result in the following status codes.

Status code
200
400
401
404
500
501

3.2.5.3.3.1 Request Body

The server MUST ignore any request body.

3.2.5.3.3.2 Response Body

No response body is returned.

3.2.5.3.3.3 Processing Details

On successful authentication the server MUST verify that [Server State].ProxyRelyingPartyTrust is set.

If it is not set the server MUST return a HTTP error code of 404.

If it is set the server MUST remove the relying party trust for the **proxy** and clear the [Server State].ProxyRelyingPartyTrust value.

3.2.6 Timer Events

None.

3.2.7 Other Local Events

None.

3.3 Proxy Registration Client Details

3.3.1 Abstract Data Model

None.

3.3.2 Timers

None.

3.3.3 Initialization

None.

3.3.4 Higher-Layer Triggered Events

None.

3.3.5 Message Processing Events and Sequencing Rules

See corresponding section on Server Details.

In all operations where the server requires authenticating the **proxy** using client TLS authentication [\[RFC2246\]](#), the proxy MUST present the certificate on [Client State].TrustCertificate during client TLS authentication.

3.3.5.1 Proxy/EstablishTrust

See corresponding section on Server Details.

3.3.5.1.1 POST

See corresponding section on Server Details.

3.3.5.1.1.1 Request Body

See corresponding section on Server Details.

3.3.5.1.1.2 Response Body

See corresponding section on Server Details.

3.3.5.1.1.3 Processing Details

[Proxy Trust].SerializedTrustCertificate MUST have an EKU for client authentication (1.3.6.1.5.5.7.3.2) ([RFC3280] section 4.2.1.13) and MUST be within validity period ([RFC1422] section 3.3). The client MUST have the private key of this certificate.

If the server response is a HTTP status code of 200 the **proxy** MUST set [Client State].TrustCertificate to [Proxy Trust].SerializedTrustCertificate for future authentication to the server.

3.3.5.2 Proxy/RenewTrust

See corresponding section on Server Details.

3.3.5.2.1 POST

See corresponding section on Server Details.

3.3.5.2.1.1 Request Body

See corresponding section on Server Details.

3.3.5.2.1.2 Response Body

See corresponding section on Server Details.

3.3.5.2.1.3 Processing Details

[Proxy Trust].SerializedReplacementCertificate MUST have an EKU for client authentication (1.3.6.1.5.5.7.3.2) ([RFC3280] section 4.2.1.13) and MUST be within validity period ([RFC1422] section 3.3). The **proxy** MUST have the private key of this certificate.

If the server response is a HTTP status code of 200 the proxy MUST set [Client State].TrustCertificate to [Proxy Trust].SerializedReplacementCertificate for future authentication to the server. <10>

3.3.5.3 Proxy/WebApplicationProxy/Trust

See corresponding section on Server Details.

3.3.5.3.1 GET

See corresponding section on Server Details.

3.3.5.3.1.1 Request Body

See corresponding section on Server Details.

3.3.5.3.1.2 Response Body

See corresponding section on Server Details.

3.3.5.3.1.3 Processing Details

No processing details.

3.3.5.3.2 POST

See corresponding section on Server Details.

3.3.5.3.2.1 Request Body

See corresponding section on Server Details.

3.3.5.3.2.2 Response Body

See corresponding section on Server Details.

3.3.5.3.2.3 Processing Details

If the server response is a HTTP status code of 200 the **proxy** MUST set [Client State].ProxyRelyingPartyTrustIdentifier to [Proxy Relying Party Trust].Identifier.

3.3.5.3.3 DELETE

See corresponding section on Server Details.

3.3.5.3.3.1 Request Body

See corresponding section on Server Details.

3.3.5.3.3.2 Response Body

See corresponding section on Server Details.

3.3.5.3.3.3 Processing Details

If the server response is a HTTP status code of 200 the **proxy** MUST clear [Client State].ProxyRelyingPartyTrustIdentifier.

3.3.6 Timer Events

None.

3.3.7 Other Local Events

None.

3.4 Service Configuration Server Details

3.4.1 Abstract Data Model

None.

3.4.2 Timers

None.

3.4.3 Initialization

None.

3.4.4 High-Layer Triggered Events

None.

3.4.5 Message Processing Events and Sequencing Rules

For the **proxy** to function properly as a proxy component on the system, it **MUST** retrieve information from the server about the service configuration and the endpoints it listens to, and about the available relying party trusts.

The following resources are required to retrieve server configuration.

Resource	Description
Proxy/GetConfiguration	Resource used to retrieve service and endpoint configuration.
Proxy/RelyingPartyTrusts	Resource used to retrieve all relying party trusts.
Proxy/RelyingPartyTrusts/{Identity}	Resource used to retrieve a particular relying party trust.

The responses to all the operations can result in the following status codes.

Status code	Description
200	The operation has succeeded.
400	The request is not valid.
401	Unauthorized for specified user credentials or for client TLS certificate.
404	The object does not exist.
405	Invalid verb used in request (GET, DELETE, POST, PUT).
409	The object already exists.
500	Version is not specified where required or any other internal error.
501	Version specified (api-version) is invalid (valid values are 1 and 2).<11>

For all operations in this section, the server requires authenticating the proxy using client TLS authentication [\[RFC2246\]](#). The server **MUST** validate that the certificate that is presented by the proxy during client TLS authentication can be validated by one of the values of [Server

State].ProxyTrustedCertificates. If the certificate cannot be validated, the server MUST return an HTTP error code of 401.

3.4.5.1 Proxy/GetConfiguration

The server MUST provide configuration for the client's run-time function.

3.4.5.1.1 GET

This operation returns a Configuration (section [2.2.2.4](#)) containing service and end-point configuration.

The operation is transported by a HTTP **GET** and can be invoked through the following URIs:

```
adfs/proxy/GetConfiguration?api-version=1
adfs/proxy/GetConfiguration?api-version=2
```

If the server is operating on AD_FS_BEHAVIOR_LEVEL_2 or higher ([\[MS-OAPX\]](#) section 3.2.1.1) and has *ServiceHostNameForUserTlsAuth* configured, it MUST return error 500 to the client for *api-version=1*.

The response message for this operation can result in the following status codes.

Status code
200
400
405
500

3.4.5.1.1.1 Request Body

The server MUST ignore any request body.

3.4.5.1.1.2 Response Body

The response body MUST be a Configuration (section [2.2.2.4](#)).

3.4.5.1.1.3 Processing Details

On successful authentication the server MUST return a [Server State].Configuration (section [3.1.1.1](#)). For requests with *api-version=2*, the Configuration can also contain *ServiceHostNameForUserTlsAuth* (section [2.2.2.4](#)).

3.4.5.2 Proxy/RelyingPartyTrusts

The **proxy** MUST retrieve information about relying party trusts to obtain relying party trust object identifiers that the proxy MUST use when identifying relying party trusts on requests to the server.

3.4.5.2.1 GET

This operation returns a Relying Party Trust List (section [2.2.2.5](#)) containing all available relying party trusts.

The operation is transported by a HTTP **GET** and can be invoked through the following URI:

```
adfs/proxy/RelyingPartyTrusts?api-version=1
```

The response message for this operation can result in the following status codes.

Status code
200
400
401
404
500
501

3.4.5.2.1.1 Request Body

The server **MUST** ignore any request body.

3.4.5.2.1.2 Response Body

The response body **MUST** be a Relying Party Trust List (section [2.2.2.5](#)).

3.4.5.2.1.3 Processing Details

On successful authentication the server **MUST** return a [Server State].RelyingPartyTrusts (section [3.1.1](#)).

3.4.5.3 Proxy/RelyingPartyTrusts/

This resource is available for the client to access data about a specific web application identified by {Identifier}.

3.4.5.3.1 GET

This operation returns a Relying Party Trust (section [2.2.2.6](#)) containing information specific to a relying party trust.

The operation is transported by a HTTP **GET** and can be invoked through the following URI:

```
adfs/proxy/RelyingPartyTrusts/{Identifier}?api-version=1
```

The response message for this operation can result in the following status codes.

Status code
200
400
401
404
500
501

3.4.5.3.1.1 Request Body

The server MUST ignore any request body.

3.4.5.3.1.2 Response Body

The response body MUST be a Relying Party Trust (section [2.2.2.6](#)).

3.4.5.3.1.3 Processing Details

On successful authentication the server MUST return a [Server State].RelyingPartyTrusts for the relying party trust with [Relying Party Trust].ObjectIdentifier equal to the URI {Identifier} value (section [3.1.1](#)).

3.4.6 Timer Events

None.

3.4.7 Other Local Events

None.

3.5 Service Configuration Client Details

3.5.1 Abstract Data Model

None.

3.5.2 Timers

None.

3.5.3 Initialization

None.

3.5.4 High-Layer Triggered Events

None.

3.5.5 Message Processing Events and Sequencing Rules

See corresponding section on Server Details.

For all operations in this section, the client MUST perform client TLS authentication [\[RFC2246\]](#) using the certificate in [Client State].TrustCertificate.

3.5.5.1 Proxy/GetConfiguration

See corresponding section on Server Details.

3.5.5.1.1 GET

See corresponding section on Server Details.

3.5.5.1.1.1 Request Body

See corresponding section on Server Details.

3.5.5.1.1.2 Response Body

See corresponding section on Server Details.

3.5.5.1.1.3 Processing Details

If the server response is a HTTP status code of 200 the **proxy** MUST set [Client State].Configuration to Configuration obtained in the response.

3.5.5.2 Proxy/RelyingPartyTrusts

See corresponding section on Server Details.

3.5.5.2.1 GET

See corresponding section on Server Details.

3.5.5.2.1.1 Request Body

See corresponding section on Server Details.

3.5.5.2.1.2 Response Body

See corresponding section on Server Details.

3.5.5.2.1.3 Processing Details

None.

3.5.5.3 Proxy/RelyingPartyTrusts/

See corresponding section on Server Details.

3.5.5.3.1 GET

See corresponding section on Server Details.

3.5.5.3.1.1 Request Body

See corresponding section on Server Details.

3.5.5.3.1.2 Response Body

See corresponding section on Server Details.

3.5.5.3.1.3 Processing Details

None.

3.5.6 Timer Events

None.

3.5.7 Other Local Events

None.

3.6 Proxy Configuration Server Details

3.6.1 Abstract Data Model

None.

3.6.2 Timers

None.

3.6.3 Initialization

None.

3.6.4 High-Layer Triggered Events

None.

3.6.5 Message Processing Events and Sequencing Rules

The **proxy** MAY use the server store to save and retrieve information about the proxy service or about applications published through the proxy. The server provides resources to set and retrieve information based on a key/value pair entry model.

The following resources are available to store custom proxy configuration on the server.

Resource	Description
Proxy/WebApplicationProxy/Store	Resource used to retrieve all entries in the store.
Proxy/WebApplicationProxy/Store/{Key}	Resource used to add, retrieve, remove, or modify an entry in the store.

The responses to all the operations can result in the following status codes.

Status code	Description
200	The operation has succeeded.
400	The request is not valid.
401	Unauthorized for the specified user credentials or for the client TLS certificate.
404	The object does not exist.
405	Invalid verb used in request (GET, DELETE, POST, PUT).
409	The object already exists.
412	A precondition failed.
500	Version is not specified where required or any other internal error.
501	Version specified (api-version) is invalid (only valid value is 1).

For all operations in this section, the server requires authenticating the proxy using client TLS authentication [\[RFC2246\]](#). The server MUST validate that the certificate that is presented by the proxy during client TLS authentication can be validated by one of the values of [Server State].ProxyTrustedCertificates. If the certificate cannot be validated, the server MUST return an HTTP error code of 401.

3.6.5.1 Proxy/WebApplicationProxy/Store

The **proxy** MAY retrieve entries from the store by means of this resource.

3.6.5.1.1 GET

This operation returns a Store Entry List (section [2.2.2.8](#)) containing all entries in the store.

The operation is transported by a HTTP **GET** and can be invoked through the following URI:

```
adfs/proxy/WebApplicationProxy/Store?api-version=1
```

The request MUST authenticate using client TLS authentication [\[RFC2246\]](#).

The response message for this operation can result in the following status codes.

Status code
200
400
401
404
409
500
501

3.6.5.1.1.1 Request Body

The server MUST ignore any request body.

3.6.5.1.1.2 Response Body

The response body MUST be a Store Entry List (section [2.2.2.8](#)).

3.6.5.1.1.3 Processing Details

Upon successful authentication the server MUST return [Server State].ProxyStore (section [3.1.1](#)).

3.6.5.2 Proxy/WebApplicationProxy/Store/

The client MAY use the store to retrieve, add, remove or modify a particular entry from the store by making requests of this resource.

3.6.5.2.1 GET

This operation returns a Store Entry (section [2.2.2.9](#)) containing its version and value.

The operation is transported by a HTTP **GET** and can be invoked through the following URI:

```
adfs/proxy/WebApplicationProxy/Store/{Key}?api-version=1
```

The request MUST authenticate using client TLS authentication [\[RFC2246\]](#).

The response message for this operation can result in the following status codes.

Status code
200
400
401
404
500
501

3.6.5.2.1.1 Request Body

The server MUST ignore any request body.

3.6.5.2.1.2 Response Body

The response body MUST be a Store Entry (section [2.2.2.9](#)).

3.6.5.2.1.3 Processing Details

Upon successful authentication the server MUST return the Store Entry (section [2.2.2.9](#)) represented by the object in [Server State].ProxyStore that has a key value with the same string value as {Key}.

If after successful authentication a Store Entry with the same string value as {Key} is not present in [Server State].ProxyStore, the server MUST return an HTTP error code of 404.

3.6.5.2.2 POST

This operation adds a new entry to the store.

The operation is transported by a HTTP **POST** and can be invoked through the following URIs:

```
adfs/proxy/WebApplicationProxy/Store/{Key}?api-version=1
adfs/proxy/PrimaryWriter/WebApplicationProxy/Store/{Key}?api-version=1
```

If the operation is invoked through `adfs/WebApplicationProxy/Store/{Key}`, the request **MUST** authenticate using client TLS authentication [\[RFC2246\]](#).

If the operation is invoked through `adfs/proxy/PrimaryWriter/WebApplicationProxy/Store/{Key}`, the request **MUST** authenticate using Integrated Windows authentication [\[RFC2478\]](#).

The response message for this operation can result in the following status codes.

Status code
200
400
401
409
500
501

3.6.5.2.2.1 Request Body

The request body is a Store Entry Key and Value (section [2.2.2.10](#)).

3.6.5.2.2.2 Response Body

No response body is returned.

3.6.5.2.2.3 Processing Details

On successful authentication the server **MUST** validate that the URI value of `{Key}` is the same as the value of `[Store Entry Key and Value].key` from the request body.

If it is not the same the server **MUST** return a HTTP error code of 400.

If it is the same the server **MUST** add the entry to the store by adding Store Entry Key and Value with a version of 1 to `[Server State].ProxyStore`.

If there is an existing value for the key specified then the server **MUST** return a HTTP error code of 409.

3.6.5.2.3 PUT

This operation modifies the value of an existing entry in the store.

The operation is transported by a HTTP **PUT** and can be invoked through the following URIs:

```
adfs/proxy/WebApplicationProxy/Store/{Key}?api-version=1
adfs/proxy/PrimaryWriter/WebApplicationProxy/Store/{Key}?api-version=1
```

If the operation is invoked through `adfs/WebApplicationProxy/Store/{Key}`, the request **MUST** authenticate using client TLS authentication [\[RFC2246\]](#).

If the operation is invoked through `adfs/proxy/PrimaryWriter/WebApplicationProxy/Store/{Key}`, the request **MUST** authenticate using Integrated Windows authentication [\[RFC2478\]](#).

The response message for this operation can result in the following status codes.

Status code
200
400
401
404
412
500
501

3.6.5.2.3.1 Request Body

The request body is a Store Entry (section [2.2.2.9](#)).

3.6.5.2.3.2 Response Body

No response body is returned.

3.6.5.2.3.3 Processing Details

On successful authentication the server **MUST** validate that the URI value of `{Key}` is the same as the value of `[Store Entry].key` from the request body.

If it is not the same the server **MUST** return a HTTP error code of 400.

If it is the same the server **MUST** find a corresponding Store Entry on `[Server State].ProxyStore` for the corresponding key.

If it is not found the server **MUST** return a HTTP error code of 404.

If it is found the server **MUST** validate that the value `[Store Entry].version` of the entry found is the same as the value of `[Store Entry].version` from the request body.

If it is not the same the server **MUST** return a HTTP error code of 412.

If it is the same the server **MUST** set the value of `[Store Entry].value` of the corresponding Store Entry on `[Server State].ProxyStore` to the `[Store Entry].value` and **MUST** increment by 1 its value of `[Store Entry].version`.

3.6.5.2.4 DELETE

This operation removes the value of an existing entry in the store.

The operation is transported by an HTTP **DELETE** and can be invoked through the following URIs:

```
adfs/proxy/WebApplicationProxy/Store/{Key}?api-version=1
adfs/proxy/PrimaryWriter/WebApplicationProxy/Store/{Key}?api-version=1
```

If the operation is invoked through `adfs/WebApplicationProxy/Store/{Key}`, the request **MUST** authenticate using client TLS authentication [\[RFC2246\]](#).

If the operation is invoked through `adfs/proxy/PrimaryWriter/WebApplicationProxy/Store/{Key}`, the request **MUST** authenticate using Integrated Windows authentication [\[RFC2478\]](#).

The response message for this operation can result in the following status codes.

Status code
200
401
404
500
501

3.6.5.2.4.1 Request Body

The server **MUST** ignore any request body.

3.6.5.2.4.2 Response Body

No response body is returned.

3.6.5.2.4.3 Processing Details

On successful authentication the server looks for a corresponding Store Entry on [Server State].ProxyStore for {Key}.

If it is not found, the server **MUST** return a HTTP error code of 404.

If it is found, the server **MUST** remove the Store Entry from [Server State].ProxyStore.

3.6.6 Timer Events

None.

3.6.7 Other Local Events

None.

3.7 Proxy Configuration Client Details

3.7.1 Abstract Data Model

None.

3.7.2 Timers

None.

3.7.3 Initialization

None.

3.7.4 High-Layer Triggered Events

None.

3.7.5 Message Processing Events and Sequencing Rules

For all operations in this section, the client MUST perform client TLS authentication [\[RFC2246\]](#) using the certificate in [Client State].TrustCertificate.

See corresponding section on Server Details.

3.7.5.1 Proxy/WebApplicationProxy/Store

See corresponding section on Server Details.

3.7.5.1.1 GET

See corresponding section on Server Details.

3.7.5.1.1.1 Response Body

See corresponding section on Server Details.

3.7.5.1.1.2 Request Body

See corresponding section on Server Details.

3.7.5.1.1.3 Processing Details

None.

3.7.5.2 Proxy/WebApplicationProxy/Store/

See corresponding section on Server Details.

3.7.5.2.1 GET

See corresponding section on Server Details.

3.7.5.2.1.1 Request Body

See corresponding section on Server Details.

3.7.5.2.1.2 Response Body

See corresponding section on Server Details.

3.7.5.2.1.3 Processing Details

None.

3.7.5.2.2 POST

See corresponding section on Server Details.

3.7.5.2.2.1 Request Body

See corresponding section on Server Details.

3.7.5.2.2.2 Response Body

See corresponding section on Server Details.

3.7.5.2.2.3 Processing Details

None.

3.7.5.2.3 PUT

See corresponding section on Server Details.

3.7.5.2.3.1 Request Body

See corresponding section on Server Details.

3.7.5.2.3.2 Response Body

See corresponding section on Server Details.

3.7.5.2.3.3 Processing Details

None.

3.7.5.2.4 DELETE

See corresponding section on Server Details.

3.7.5.2.4.1 Request Body

See corresponding section on Server Details.

3.7.5.2.4.2 Response Body

See corresponding section on Server Details.

3.7.5.2.4.3 Processing Details

None.

3.7.6 Timer Events

None.

3.7.7 Other Local Events

None.

3.8 Application Publishing Server Details

3.8.1 Abstract Data Model

None.

3.8.2 Timers

None.

3.8.3 Initialization

None.

3.8.4 High-Layer Triggered Events

None.

3.8.5 Message Processing Events and Sequencing Rules

The following resources are available to set the publishing settings to web applications.

Resource	Description
Proxy/RelyingPartyTrusts/{Identity}/PublishedSettings	Resource used to publish a particular web application through the client.

The responses to all the operations can result in the following status codes.

Status code	Description
200	The operation has succeeded.
400	The request is not valid.
401	Unauthorized for the specified user credentials or for the client TLS certificate.
404	The object does not exist.
405	Invalid verb used in request (GET, DELETE, POST, PUT).
409	The object already exists.
500	Version is not specified where required or any other internal error.
501	Version specified (api-version) is invalid (only valid value is 1).

For all operations in this section, the server requires authenticating the **proxy** using client TLS authentication [\[RFC2246\]](#). The server MUST validate that the certificate that is presented by the proxy during client TLS authentication can be validated by one of the values of [Server State].ProxyTrustedCertificates. If the certificate cannot be validated, the server MUST return an HTTP error code of 401.

If the operation authenticates using Integrated Windows authentication [\[RFC2478\]](#), the server MUST validate that the authenticated principal is authorized to do the corresponding operation on the server.

3.8.5.1 Proxy/RelyingPartyTrusts/{Identifier}/PublishedSettings

3.8.5.1.1 POST

This operation creates a new set of publishing settings on a relying party trust.

The operation is transported by a HTTP **POST** and can be invoked through the following URIs:

```
adfs/proxy/RelyingPartyTrusts/{Identifier}/PublishedSettings?api-version=1
adfs/proxy/PrimaryWriter/RelyingPartyTrusts/{Identifier}/PublishedSettings?api-version=1
```

If the operation is invoked through

adfs/proxy/RelyingPartyTrusts/{Identifier}/PublishedSettings?api-version=1, the request MUST authenticate using client TLS authentication [\[RFC2246\]](#).

If the operation is invoked through

adfs/proxy/PrimaryWriter/RelyingPartyTrusts/{Identifier}/PublishedSettings?api-version=1, the request MUST authenticate using Integrated Windows authentication [\[RFC2478\]](#).

The response message for this operation can result in the following status codes.

Status code
200
400
401
404
409
500
501

3.8.5.1.1.1 Request Body

The request body MUST be a Relying Party Trust Publishing Settings (section [2.2.2.7](#)).

3.8.5.1.1.2 Response Body

No response body is returned.

3.8.5.1.1.3 Processing Details

If the publishing settings specified in Relying Party Trust Publishing Settings have been set previously on [Server State].RelyingPartyTrusts, the server MUST return a HTTP error code of 409.

If they have not been set the server MUST add the Relying Party Trust Publishing Settings for the relying party trust identifier with {Identifier}. The server MUST add a new URL to [Server State].RelyingPartyTrusts[objectIdentifier:={Identifier}].proxyTrustedEndpoints with the value of [Relying Party Trust Publishing Settings].proxyTrustedEndpoint and add a new mapping to [Server State].RelyingPartyTrusts[objectIdentifier:={Identifier}].proxyEndpointMappings with the value of [Relying Party Trust Publishing Settings].internalURL to Key and [Relying Party Trust Publishing Settings].externalURL to Value.

3.8.5.1.2 DELETE

This operation removes the publishing settings for a relying party trust.

The operation is transported by a HTTP **DELETE** and can be invoked through the following URIs:

```
adfs/proxy/RelyingPartyTrusts/{Identifier}/PublishedSettings?api-version=1
adfs/proxy/PrimaryWriter/RelyingPartyTrusts/{Identifier}/PublishedSettings?api-version=1
```

If the operation is invoked through

adfs/proxy/RelyingPartyTrusts/{Identifier}/PublishedSettings?api-version=1, the request MUST authenticate using client TLS authentication [\[RFC2246\]](#).

If the operation is invoked through

adfs/proxy/PrimaryWriter/RelyingPartyTrusts/{Identifier}/PublishedSettings?api-version=1, the request MUST authenticate using Integrated Windows authentication [\[RFC2478\]](#).

The response message for this operation can result in the following status codes.

Status code
200
400
401
404
500
501

3.8.5.1.2.1 Request Body

The request body MUST be a Relying Party Trust Publishing Settings (section [2.2.2.7](#)).

3.8.5.1.2.2 Response Body

No response body is returned.

3.8.5.1.2.3 Processing Details

If the publishing settings specified in Relying Party Trust Publishing Settings have not been set previously the server MUST return a HTTP error code of 404.

If they have been set then use the following algorithm for processing this request:

1. If the [Relying Party Trust Publishing Settings].proxyTrustedEndpoint is missing or the [Relying Party Trust Publishing Settings].internalUrl is present in the request body, the server MUST return an HTTP error code of 400.
2. If the Relying Party Trust (section [2.2.2.6](#)) with objectIdentifier with the same string value as {Identifier} in [Server State].RelyingPartyTrusts is not found, or if [Server State].RelyingPartyTrusts[objectIdentifier:={Identifier}].proxyTrustedEndpoints with the value of [Relying Party Trust Publishing Settings].proxyTrustedEndpoint is not found, or if [Relying Party Trust Publishing Settings].externalUrl is specified and an entry with the matching externalUrl is not found in [Server State].RelyingPartyTrusts[objectIdentifier:={Identifier}].proxyEndpointMappings, the server MUST return an HTTP error code of 404.
3. On the Relying Party Trust (section [2.2.2.6](#)) [Server State].RelyingPartyTrusts[objectIdentifier:={Identifier}], remove the entry from Relying Party Trust Publishing Settings (section [2.2.2.7](#)) that contains in proxyTrustedEndpoint the value of [Relying Party Trust Publishing Settings].proxyTrustedEndpoint from the request body.
4. If the value of [Relying Party Trust Publishing Settings].externalUrl is present in the request body, remove the entry from [Server State].RelyingPartyTrusts[objectIdentifier:={Identifier}].proxyEndpointMappings that has an externalUrl matching of [Relying Party Trust Publishing Settings].externalUrl from the request body.

3.8.6 Timer Events

None.

3.8.7 Other Local Events

None.

3.9 Application Publishing Client Details

3.9.1 Abstract Data Model

None.

3.9.2 Timers

None.

3.9.3 Initialization

None.

3.9.4 High-Layer Triggered Events

None.

3.9.5 Message Processing Events and Sequencing Rules

See corresponding section on Server Details.

In all operations where the server requires authenticating the client using client TLS authentication [RFC2246], the client MUST perform client TLS authentication [RFC2246] using the certificate in [Client State].TrustCertificate.

3.9.5.1 Proxy/RelyingPartyTrusts/{Identifier}/PublishedSettings

See corresponding section on Server Details.

3.9.5.1.1 POST

See corresponding section on Server Details.

3.9.5.1.1.1 Request Body

See corresponding section on Server Details.

3.9.5.1.1.2 Response Body

See corresponding section on Server Details.

3.9.5.1.1.3 Processing Details

If the server response is a HTTP status code of 200 the **proxy** MUST add a new identifier object to [Client State].RelyingPartyTrusts with the RelyingPartyTrust.Identifier set to {Identifier}.

3.9.5.1.2 DELETE

See corresponding section on Server Details.

3.9.5.1.2.1 Request Body

See corresponding section on Server Details.

3.9.5.1.2.2 Response Body

See corresponding section on Server Details.

3.9.5.1.2.3 Processing Details

If the server response is a HTTP status code of 200 the **proxy** MUST remove from [Client State].RelyingPartyTrusts the object with RelyingPartyTrust.Identifier with the same string value as {Identifier}.

3.9.6 Timer Events

None.

3.9.7 Other Local Events

None.

3.10 Proxy Runtime Behaviors Server Details

3.10.1 Abstract Data Model

None.

3.10.2 Timers

None.

3.10.3 Initialization

None.

3.10.4 High-Layer Triggered Events

None.

3.10.5 Message Processing Events and Sequencing Rules

The following resource is available to send a request along with the certificate to the server.

Resource	Description
BackEndProxyTLS	Resource used to obtain a request along with the certificate used for client TLS authentication [RFC2246] .

The responses to all the operations can result in the following status codes.

Status code	Description
200	The operation has succeeded.
400	The request is not valid.
401	Unauthorized for client TLS certificate.
500	Internal error.

For all operations in this section, the server requires authenticating the **proxy** using client TLS authentication [\[RFC2246\]](#). The server MUST validate that the certificate that is presented by the client during client TLS authentication can be validated by one of the values of [Server State].ProxyTrustedCertificates. If the certificate cannot be validated, the server MUST return an HTTP error code of 401.

3.10.5.1 BackEndProxyTLS

The **proxy** MUST support client TLS authentication [\[RFC2246\]](#) on behalf of the server by obtaining the certificate and forwarding it along with the receiving message to the server.

3.10.5.1.1 POST

This operation obtains a request along with a certificate.

The operation is transported by a HTTP **POST** and can be invoked through the following URI:

The server requires authenticating the client using client TLS authentication [\[RFC2246\]](#).

The response message for this operation can result in the following status codes.

Status code
200
400
401
500

3.10.5.1.1.1 Request Body

The request body MUST be a base64url encoded ([\[RFC4648\]](#) section 5) Serialized Request with Certificate (section [2.2.2.11](#)).

3.10.5.1.1.2 Response Body

The response from the server MUST be returned to the client.

3.10.5.1.1.3 Processing Details

The server MUST treat [Serialized Request with Certificate].SerializedClientCertificate as the certificate of the end-user, and SHOULD assume that the client has already verified the original requester's proof of possession of the private key corresponding to that certificate.

The server MUST process the request as if it was received directly to the endpoint in the server as specified in the request.

If [Serialized Request with Certificate].ErrorType is set to 1 ("Certificate") and [Serialized Request with Certificate].ErrorCode is set to non-zero, then the server SHOULD fail the client's request.

3.10.6 Timer Events

None.

3.10.7 Other Local Events

None.

3.11 Proxy Runtime Behaviors Client Details

3.11.1 Abstract Data Model

None.

3.11.2 Timers

None.

3.11.3 Initialization

None.

3.11.4 High-Layer Triggered Events

None.

3.11.5 Message Processing Events and Sequencing Rules

The client SHOULD listen for HTTP requests based on the server characteristics in [Client State].Configuration.

For each object, CurrentEndpointConfiguration in [Client State].Configuration.EndpointConfiguration, the client SHOULD do the following:

1. Listen for HTTP requests whose URLs conform to the following rules:
 1. HostName of the URL is one of the following:
 - [Client State].Configuration.ServiceConfiguration.ServiceHostName
 - [Client State].Configuration.ServiceConfiguration.ServiceHostNameForUserTlsAuth
 - "EnterpriseRegistration.<PossibleUpnSuffix>" where <PossibleUpnSuffix> is one of either [Client State].Configuration.ServiceConfiguration.DiscoveredUpnSuffixes or [Client State].Configuration.ServiceConfiguration.CustomUpnSuffixes.
 2. If CurrentEndpointConfiguration.PortType is 0 ("HttpPort"), the port component of the URL is [ServiceConfiguration.HttpPort].
 3. If CurrentEndpointConfiguration.PortType is 1 ("HttpsPort"), the port component of the URL is [ServiceConfiguration.HttpsPort].
 4. If CurrentEndpointConfiguration.PortType is 2 ("HttpsPortForUserTlsAuth"), the port component of the URL is [ServiceConfiguration.HttpsPortForUserTlsAuth].
 5. The Path component of the URL is a subpath of [CurrentEndpointConfiguration.Path].
2. If CurrentEndpointConfiguration.ClientCertificateQueryMode is 1 ("QueryAndAccept") and the request does not have a public key authentication header or user agent indicator ([\[MS-PKAP\]](#) section 3.1.5.1.1), then the client SHOULD attempt to retrieve end-user X509 certificate [\[RFC4158\]](#) using client **TLS** authentication [\[RFC2246\]](#). If it obtains a certificate the client MUST follow processing in section [3.11.5.1](#).
3. If CurrentEndpointConfiguration.ClientCertificateQueryMode is 2 ("QueryAndRequire"), then the client SHOULD attempt to retrieve end-user X509 certificate [\[RFC4158\]](#) using client TLS authentication [\[RFC2246\]](#). If it obtains a certificate, the client MUST follow the processing in section 3.11.5.1.
4. If CurrentEndpointConfiguration.SupportsNtlm is true, the client SHOULD ensure that SPNEGO-based authentication requests [\[RFC4559\]](#) with the "Negotiate" auth-scheme are converted to NTLM.
5. If the configuration field **IgnoreTokenBinding** is not set to True (section [2.2.2.4](#)), the configuration field **UpdatedFarmBehaviorLevel** is greater than or equal to AD_FS_BEHAVIOR_LEVEL_4 ([\[MS-OAPX\]](#) section 3.2.1.1), and the end-user request contains token binding information in the form of the Sec-Token-Binding header defined in [\[IETF-DRAFT-TOKBIND-H\]](#) section 2, the client SHOULD construct a signed header using the structure defined in section [2.2.1.6](#), which is a serialized **JWT**. The client then performs the following steps:

1. Token binding information on the request is parsed into the provided_token_binding and referred_token_binding structures, as defined in [IETFDRAFT-TOKBIND-H] section 2 and [IETFDRAFT-TOKBINDPROT] section 3.1.
 2. The provided_token_binding information is included as a claim in the JWT, with claim name "Sec-Provided-Token-Binding-ID".
 3. The referred_token_binding information is included as a claim in the JWT, with claim name "Sec-Referred-Token-Binding-ID".
6. If no certificate was obtained in step 2, or if a certificate was obtained in steps 2 or 3, but the section 3.11.5.1 validation fails when the CurrentEndpointConfiguration.CertificateValidation value is 2 ("IssuedByDrs"), then the client SHOULD replay the request as follows:
1. The request SHOULD be made to the following URL:
 1. If CurrentEndpointConfiguration.ServicePortType is 0, then form the URL as "http://[ServiceConfiguration.ServiceHostName]:[ServiceConfiguration.HttpPort]/[CurrentEndpointConfiguration.ServicePath]".
 2. If CurrentEndpointConfiguration.ServicePortType is 1, then form the URL as "https://[ServiceConfiguration.ServiceHostName]:[ServiceConfiguration.HttpsPort]/[CurrentEndpointConfiguration.ServicePath]".
 3. If CurrentEndpointConfiguration.ServicePortType is 2, then form the URL as "https://[ServiceConfiguration.ServiceHostName]:[ServiceConfiguration.HttpsPortForUserTlsAuth]/[CurrentEndpointConfiguration.ServicePath]".
 2. The client SHOULD add the headers in section 2.2.1 to the request.
7. If no certificate was obtained in step 3, then the client SHOULD<12> perform the following steps:
1. The client constructs a request as in section 3.10.5.1 with [Serialized Request with Certificate] set to following values:
 - [Serialized Request with Certificate].ErrorType MUST be set to 1 ("Certificate").
 - [Serialized Request with Certificate].ErrorCode MUST be set to 1168.
 2. The client then performs the common processing defined in section 3.11.5.2.

3.11.5.1 End-user X509 Certificate Success Processing

If the client obtains a certificate of the end-user then the client SHOULD validate the X509 certificate [RFC4158] based on the CurrentEndpointConfiguration.CertificateValidation.

- If the CurrentEndpointConfiguration.CertificateValidation value is 0 ("None") then no validation SHOULD be performed.
- If the CurrentEndpointConfiguration.CertificateValidation value is 1 ("Ssl") then the whole chain validation [RFC4158] of the certificate SHOULD be performed.
- If the CurrentEndpointConfiguration.CertificateValidation value is 2 ("IssuedByDrs") then the client SHOULD validate that the end-user certificate was issued by one of ServiceConfiguration.DeviceCertificateIssuers.

If the validation of the end-user certificate was successful, or if the validation of the end-user certificate failed and the CurrentEndpointConfiguration.CertificateValidation value is 1, the following processing occurs:

- The client MUST construct a request as in section 3.10.5.1.

- If the validation of the end-user certificate was successful, then the [Serialized Request with Certificate].SerializedClientCertificate MUST be set to the base64 string encoded ([\[RFC4648\]](#) section 4) X509 certificate [RFC4158]. Otherwise, the [Serialized Request with Certificate].ErrorType SHOULD be set to 1 ("Certificate") and the [Serialized Request with Certificate].ErrorCode SHOULD be set to the error value that was encountered while validating the end-user certificate. [<13>](#)
- The client then performs the common processing defined in section 3.11.5.2.

If the validation of the end-user certificate failed and the CurrentEndpointConfiguration.CertificateValidation value is 2, the client SHOULD replay the request as defined in section 3.11.5 step 6.

3.11.5.2 End-user X509 Certificate Common Processing

If CurrentEndpointConfiguration.CertificateValidation value is 2 ("IssuedByDrs") then the [Serialized Request with Certificate].CertificateUsage MUST be set to 2 ("Device").

If CurrentEndpointConfiguration.CertificateValidation value is 1 ("Ssl") then the [Serialized Request with Certificate].CertificateUsage MUST be set to 1 ("User").

The [Serialized Request with Certificate].Request elements values SHOULD be copied from the incoming HTTP request.

The request SHOULD be made to `https://[ServiceConfiguration.ServiceHostName]:[ServiceConfiguration.HttpsPort]/adfs/backendproxyservlet` and the client MUST authenticate with client **TLS** [\[RFC2246\]](#) using [Client State].TrustCertificate.

3.11.6 Timer Events

None.

3.11.7 Other Local Events

None.

3.12 Application Proxy Runtime Behaviors Server Details

3.12.1 Abstract Data Model

None.

3.12.2 Timers

None.

3.12.3 Initialization

None.

3.12.4 High-Layer Triggered Events

None.

3.12.5 Message Processing Events and Sequencing Rules

3.12.5.1 Issue Preauthentication

The server MUST implement the behaviors in this section if and only if the following is met for a particular incoming request:

1. The request contains the header X-MS-Proxy, as defined in section [2.2.1.1](#).
2. The [Server State].ProxyRelyingPartyTrust, as defined in section [3.1.1.1](#), that has the same URI {web-application-for-client-id} (using a case-insensitive comparison) as an object in the [Server State].RelyingPartyTrust array, as defined in section [2.2.2.5](#), has the enabled property set to true.
3. The [Relying Party Trust] being preauthenticated exists and has the value of publishedThroughProxy set to true. Note that **pre-authentication** is different for each protocol; refer to subsequent sections for details.

3.12.5.1.1 Proxy Preauthentication

This operation processes a request for authentication and returns a **proxy token** as described in section [3.13.5.1](#) upon success.

The operation is transported by a HTTP **GET** and can be invoked through the following URI:

```
adfs/ls?version=1.0&action=signin&realm={web-application-for-client-id}&apprealm={web-application-id}&returnurl={client-url-to-issue-token}
```

The response message for this operation can result in the following status codes.

Status code	Description
200	The operation has succeeded.
403	The access is forbidden.
500	Internal error.

3.12.5.1.1.1 Request Body

The server MUST ignore any request body.

3.12.5.1.1.2 Response Body

No response body is returned.

3.12.5.1.1.3 Processing Details

The server MUST validate that {web-application-for-client-id} corresponds to the value of [Server State].ProxyRelyingPartyTrust.objectIdentifier. If validation fails, the server MUST return a HTTP error code of 500.

The server MUST validate that the request meets the conditions to issue **pre-authentication** (section [3.12.5.1](#)) for the web application in [Server State].RelyingPartyTrusts with objectIdentifier equal to {web-application-id}.

The server MUST validate that the Relying Party Trust (section [2.2.2.6](#)) proxyTrustedEndpoints contains a URL with a scheme, host and port that match those of {client-url-to-issue-token} and that prefix-matches the url-path of {client-url-to-issue-token} (for URL components see [\[RFC1738\]](#) sections 2.1 and 3.1). If validation fails, the server MUST return a HTTP error code of 500.

The server performs authentication of the request based on the server's authentication policy for [Server State].ProxyRelyingPartyTrust. If authentication fails the server MUST return a HTTP error code of 403.

If authentication succeeds the server MUST return a HTTP status code of 302 with a base64url encoded ([\[RFC4648\]](#) section 5) **proxy token** (section [3.13.5.1](#)) in the URL query string parameter "authToken".

3.12.5.1.2 SAML-P Extensions for Preauthentication

The server MUST validate that the request meets the conditions to issue **pre-authentication** (section [3.12.5.1](#)) for the web application in [Server State].RelyingPartyTrusts with identifiers containing a string value that matches the <Issuer> element value ([\[SAMLCore2\]](#) section 2.2.5) in the <AuthnRequest> element ([SAMLCore2]).

Upon successful authentication ([SAMLCore2] section 3.4.1.4) the server MUST do the following before sending the response to the response URL:

1. Transform the response URL based on the values of [Relying Party Trust].proxyEndpointMappings for the web application by replacing the response URL string portion that matches the Key value (internal URL mapping value) with the value of Value (external URL mapping value). If there is no match the response URL MUST not be changed.
2. If the request is an IdP initiated request the server MUST perform authentication of the request based on the server's authentication policy for [Server State].ProxyRelyingPartyTrust. If authentication fails the server MUST respond according to [SAMLCore2] defined behavior for failed authentication.
3. If authentication succeeds the server MUST include in the response URL a query string parameter with name "authToken" with a value of a base64url encoded ([\[RFC4648\]](#) section 5) **proxy token** (section [3.13.5.1](#)).

The server MUST send the response to the response URL.

3.12.5.1.3 WS-Fed Extensions for Preauthentication

If the server implements [\[WSFederation1.2\]](#) then the server MUST implement the following processing.

The server MUST validate that the request meets the conditions to issue **pre-authentication** (section [3.12.5.1](#)) for the web application in [Server State].RelyingPartyTrusts with identifiers containing a string value that matches the wtrealm query string parameter value.

Upon successful authentication ([WSFederation1.2] section 13.1.1) the server MUST do the following before sending the response to the response URL:

1. Transform the response URL based on the values of [Relying Party Trust].proxyEndpointMappings for the web application by replacing the response URL string portion that matches the Key value (internal URL mapping value) with the value of Value (external URL mapping value). If there is no match the response URL MUST not be changed.
2. If pre-authentication has not happened yet [<14>](#) the server MUST perform authentication of the request based on the server's authentication policy for [Server State].ProxyRelyingPartyTrust. If authentication fails the server MUST respond according to [WSFederation1.2] defined behavior for failed authentication.

3. If authentication succeeds the server MUST include in the response URL a query string parameter with name "authToken" with a value of a base64url encoded ([RFC4648] section 5) **proxy token** (section 3.13.5.1).

The server MUST send the response to the response URL.

3.12.5.1.4 OAuth Extensions for Preauthentication

If the server implements the OAuth 2.0 Protocol Extensions [MS-OAPX], then the server MUST implement the following behaviors.

The server MUST validate that the request meets the conditions to issue **pre-authentication** (section 3.12.5.1) for the web application in [Server State].RelyingPartyTrusts with identifiers containing a URI matching the "resource" query string parameter value.

Upon successful authentication [MS-OAPX], the server MUST do the following before sending the response.

The server performs authentication of the request based on the server's authentication policy for [Server State].ProxyRelyingPartyTrust. If authentication fails the server MUST respond according to [MS-OAPX] defined behavior for failed authentication.

If authentication succeeds the server MUST generate a **proxy token** (section 3.13.5.1). The server MUST take the proxy token and combine it with the token targeted for the application in a [Combined Token] (section 2.2.2.19) and base64url encode ([RFC4648] section 5) the results. The server MUST use this [Combined Token] in all references to "token" in [MS-OAPX].

3.12.5.1.5 Proxy Preauthentication for Active Clients

This operation processes a request for authentication, and returns a **proxy token** as described in section 3.13.5.1 upon success.<15>

The operation is transported by a HTTP POST and can be invoked through the following URI:

```
adfs/proxy/relyingpartytoken?api-version=1
```

The response message for this operation can result in the following status codes.

Status code	Description
200	The operation has succeeded.
400	The request is not valid.
401	Unauthorized for client TLS certificate.
405	Invalid verb used in request (GET, DELETE, PUT).
500	Internal error.
501	The version specified (api-version) is invalid. The only valid value is 1.

For this operation, the server requires authenticating the proxy using client TLS authentication [RFC2246]. The server MUST validate that the certificate that is presented by the proxy during client TLS authentication can be validated by one of the values of [Server State].ProxyTrustedCertificates. If the certificate cannot be validated, the server MUST return an HTTP error code of 401.

3.12.5.1.5.1 Request Body

The request body MUST be an [Authentication Request](#) complex type (section 2.2.2.21).

3.12.5.1.5.2 Response Body

The response body MUST be a [Proxy Token Wrapper](#) complex type (section 2.2.2.20) if processing was successful. If processing was not successful and the status code is 400, 401, 500, or 501, the response body can be an [Error Response](#) complex type (section 2.2.2.22), but this is not required. The response body MUST be empty in all other cases.

3.12.5.1.5.3 Processing Details

The server MUST validate that {web-application-for-client-id} corresponds to the value of [Server State].ProxyRelyingPartyTrust.objectIdentifier. If validation fails, the server MUST return an HTTP error code of 500.

The server MUST validate that the request meets the conditions to issue **pre-authentication** (section [3.12.5.1](#)) for the web application in [Server State].RelyingPartyTrusts with objectIdentifier equal to {web-application-id}.

The server performs authentication of the request based on the server's authentication policy for [Server State].ProxyRelyingPartyTrust. If authentication fails, the server MUST return an HTTP error code of 403.

If authentication succeeds, the server MUST return an HTTP status code of 200 with a [Proxy Token Wrapper](#) complex type (section 2.2.2.20) in the response body.

3.12.6 Timer Events

None.

3.12.7 Other Local Events

None.

3.13 Application Proxy Runtime Behaviors Client Details

3.13.1 Abstract Data Model

None.

3.13.2 Timers

None.

3.13.3 Initialization

None.

3.13.4 High-Layer Triggered Events

None.

3.13.5 Message Processing Events and Sequencing Rules

On receiving any request the client needs to identify if the request is preauthenticated to either allow the access or initiate **pre-authentication**.

3.13.5.1 Preauthentication

A request is preauthenticated if it contains a [Proxy Token] (section [2.2.2.18](#)) signed using JSON Web Signature (JWS) [\[IETF DRAFT-JWS\]](#) with the signing certificate published by the server through the Federation Metadata [\[WSFederation1.2\]](#).

Once a request has been identified as preauthenticated, the **proxy** MUST allow access by replaying the request to the corresponding internal address without the [Proxy Token].

Other claims might be present as name/value pairs depending on the issuance rules for the proxy configured at the server. It is left to the proxy implementer as to how to use these claims.

3.13.5.1.1 Query String Based Preauthentication

The request is preauthenticated if it contains a valid base64url encoded ([\[RFC4648\]](#) section 5) **proxy token** (section [3.13.5.1](#)) from the server on the query string parameter "authToken". The token is validated according to section 3.13.5.1.

After successful **pre-authentication** the proxy MUST remove the authToken parameter with its value before replaying the request to the internal URL.

3.13.5.1.2 HTTP Authorization Header Based Preauthentication

If the request contains a HTTP Authorization header with a valid base64URL encoded ([\[RFC4648\]](#) section 5) [Combined Token] (section [2.2.2.19](#)) then request can be preauthenticated by validating [Combined Token].proxy_token as in section [3.13.5.1](#).

The client MUST use [Combined Token].proxy_token to authorize the access to the web application.

After successful **pre-authentication** the client MUST replace the HTTP Authorization header value with a base64URL encoded ([\[RFC4648\]](#) section 5) value of [Combined Token].access_token before replaying the request to the internal URL.

3.13.5.2 Initiate Preauthentication

If the request does not contain a **proxy token** then the request is unauthenticated and the client MUST initiate **pre-authentication**.

If the client is servicing a request for the application identified by one of the entries in [Client State].RelyingPartyTrusts then the client MUST initiate pre-authentication as follows:

1. If [Relying Party Trust State].RedirectBasedPreauth is "true" then the client MUST follow processing rules in section [3.13.5.2.1](#).
2. If [Relying Party Trust State].RedirectBasedPreauth is "false" then the client MUST follow processing rules in section [3.13.5.2.2](#).

3.13.5.2.1 Initiate Redirect-based Preauthentication

Once a request to a web application has been identified as unauthenticated, the **proxy** MUST initiate **pre-authentication** by returning a HTTP 307 Temporary Redirect message to the client, redirecting the client to the following server end-point URL:

```
"https://" + [Client State].Configuration.ServiceConfiguration.ServiceHostName + ":" +  
[Client State].Configuration.ServiceConfiguration.HttpsPort + "/adfs/ls"
```

The redirect URL MUST have the following query string parameters.

Parameter	Value
version	Version of the protocol. It MUST be "1.0".
action	Action on authentication request. It MUST be "signin".
realm	Identifier for the Proxy Relying Party Trust. It MUST be [Client State].ProxyRelyingPartyTrustIdentifier (section 3.1.1.2).
apprealm	objectIdentifier of the application being accessed (section 2.2.2.6).
returnurl	URL of the incoming request.

3.13.5.2.2 Response to [MS-OFBA] Requests

Once a request to a web application has been identified as unauthenticated, the **proxy** MUST initiate **pre-authentication**. To do this the proxy MUST identify whether the request is from a Microsoft Office application that relies on the Office Forms Based Authentication (OFBA) Protocol [\[MS-OFBA\]](#).

To identify requests from Microsoft Office clients to application services relying on the OFBA protocol, the proxy MUST check if the request is an HTTP OPTIONS with a particular value on the User-Agent HTTP header or with a particular value on the X-Forms_Based_Auth_Accepted HTTP header (any of them):

Header	Value
User-Agent	Any of the following: "Microsoft Data Access Internet Publishing Provider" "Microsoft-WebDAV-MiniRedir" "non-browser" "MSOffice ##" where ## is an integer number "MSOffice XXXX ##" where XXXX is a value of "Word", "Excel", "PowerPoint" and "OneNote" and ## is an integer number "Mozilla/4.0 (compatible; MS FrontPage)" "Microsoft Office Protocol Discovery"
X-Forms_Based_Auth_Accepted	Any of the following: "t"

If the request is from a Microsoft Office client relying on the OFBA protocol, the server MUST return an HTTP error code of 403 to the client with the following headers:

Header	Value				
X-Forms_Based_Auth_Required	URL for the sign-in request: <table><tr><th>Parameter</th><th>Value</th></tr><tr><td>version</td><td>Version of the protocol. It MUST be "1.0".</td></tr></table>	Parameter	Value	version	Version of the protocol. It MUST be "1.0".
Parameter	Value				
version	Version of the protocol. It MUST be "1.0".				

Header	Value	
	action	Action on authentication request. It MUST be "signin".
	realm	Identifier for the Proxy Relying Party Trust. It MUST be [Client State].ProxyRelyingPartyTrustIdentifier (section 3.1.1.2).
	aprealm	objectIdentifier of the application being accessed (section 2.2.2.6).
	returnurl	URL of the incoming request.
X-Forms_Based_Auth_Return_Url	URL of incoming request.	

For requests from non-Microsoft-Office clients accessing services that implement the OFBA protocol [MS-OFBA] that rely on **AD FS** for authentication, the proxy MUST return an HTTP error code of 401 Unauthorized with the following header.

Header	Value
WWW-Authenticate	"Bearer authorization_uri=https://" + [Client State].Configuration.ServiceConfiguration.ServiceHostName + ":" + [Client State].Configuration.ServiceConfiguration.HttpsPort + "/adfs/oauth2/authorize"

3.13.5.2.3 Response to Active Requests

The **proxy** MAY choose to preauthenticate requests by making backend requests to the server as specified in section [3.12.5.1.5](#), provided the proxy deems that the request contains the credentials it needs to be preauthenticated. [<16>](#)

The proxy MUST perform client TLS authentication [\[RFC2246\]](#) using the certificate in [Client State].TrustCertificate.

- If the request contains Username and Password in the Authorization header as specified in [RFC2617], they are used as **username** and **password** in the [Authentication Request](#) (section 2.2.2.21).
- If the request was made using SSL mutual authentication [\[RFC6101\]](#), the client certificate SHOULD be identified by the proxy as whether it is the proof of the device or the proof of the user.
 - If the client certificate is the proof of the user, it is used as the **userCertificate** in the Authentication Request (section 2.2.2.21).
 - If the client certificate is the proof of the device, it is used as the **deviceCertificate** in the Authentication Request (section 2.2.2.21).
- Any HTTP headers from the incoming request are passed on to the server as the **httpHeaders** in the Authentication Request (section 2.2.2.21).

If the **pre-authentication** request resulted in an error, the proxy MUST send HTTP 401 to the client.

If the pre-authentication request returned a valid response as specified in section [3.12.5.1.5.2](#), the value of **authToken** in the [Proxy Token Wrapper](#) (section 2.2.2.20) is used for pre-authentication according to the rules specified in section [3.13.5.1](#).

The proxy MAY allow the "Authorization" header from the incoming HTTP request [RFC2617], to propagate to the backend application.

3.13.6 Timer Events

None.

3.13.7 Other Local Events

None.

4 Protocol Examples

4.1 Establishing Proxy Trust with the Server

4.1.1 Client Request

```
POST https://sts1.contoso.com/adfs/Proxy/EstablishTrust HTTP/1.1
Content-Type: application/json
Authorization: Basic YWRtaW5pc3RyYXRvcjBZHZjlbWJsZUA2
Host: sts1.contoso.com
Content-Length: 2388
Expect: 100-continue
```

```
{ "SerializedTrustCertificate": "MIIG0zCCBLugAwIBAgITOGAAAAWDWt3Svu3yfgAAAAABTANBgkqhkiG9w0BAQsFADAYMRyWfAYDVQQDEw1tdWFsaWRmdDI3LUNBMB4XDTEzMDcxMjIzMDgxNVoXDTE0MDcxMjIzMDgxNVowbjETMBEGCgmS
JomT8ixkARKWA2NvbTETMBEGCgmS
JomT8ixkARKWA2RmdDEaMBGCGmS
JomT8ixkARKWCm1lYWxpZGZ0MjcxDjAMBGNV
BAMTBVVzZXJzMRyWfAYDVQQDEw1BZG1pbmlzdHJhdG9yMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAsFeNg
BQ9p6c6c9yGeXx9g6TavGJHnAn5hlKTHglBAh\1md00+Fcn2QD8RB2yWu8kh4uXSUwc2VLAAbM095M35o\U0uh1kJODf
bpOu3KL7rufPMeDUHtLNIxyL91gRxoBEPEKv8okMKmQtQE4DgpY5yFiL3G0EGM4S\QOZxhiztKP9\ne6PEu\rMrdc6
8FoxG+6Hwp3WRgYrV+C5\7UsD5LlMMWxzXm4TDPtjebvcFS9WKD9wd89sEUpvomRQglLj+sXsS\DVpo8IhbbmYSzn6f
\WESRKRyJoUDBWymIGj4CA5mgDvjtbEiawC7YDv4E2i8H2RVGtldJtweoeWs2ij5QIDAQABO4ICvjCCARowFwYJKwYBb
AGCNxQCBAoeCABVAHMAZQByMcKGA1UdJQQiMCAGCisGAQQBgjkAwGCCsGAQUFBwMEBggrBgEFBQcDAjAOBgNVHQ8BAf
8EBAMCBaAwRAYJKoZIhvcNAQkPBDCwNTAObggqhkiG9w0DAGICAIAwDgYIKoZIhvcNAwQCAgCAMAcGBSS0AwIHMAoGCCq
GSIB3DQMHMB0GA1UdDgQWBBRmTBByGxQUzt2gjmhuCZrVLai65TAfBgNVHSMEGDAWgBRdxEDXM6dBSWx2luJ+kQ2tiLr4
GDCB1AYDVR0fBlHMMIHJMIHGoIHDoIAHoG9bGRhcDovLy9DTj1tdWFsaWRmdDI3LUNBLENOPW1kZnRkYyxDTj1DRFAsQ
049UHVibGljJTIwS2V5JTIwU2VydmljZXMsQ049U2VydmljZXMsQ049Q29uZmlndXJhdGlvbixEQz1tdWFsaWRmdDI3LE
RDPWRmdCxEQz1jb20\Y2VydGlmawNhdGVSSXZvY2F0aW9uTGlzdD9iYXNlP29iamVjdENsYXNzPWNSTERpc3RyaWJldG
lvb1bVaw50MIHJBggrBgEFBQcBAQSBvDCBuTCBtgYIKwYBBQUHMAKGalsZGFwOi8vL0NOPW1lYWxpZGZ0MjcjctQ0EsQ04
9QU1BLENOPVB1YmxpYyUyMETleSUyMFNlcnZpY2VzLENOPVNlcnZpY2VzLENOPUNvbmZpZ3VyYXRpb24sREM9bXVhbGlk
ZnQyNyxEQz1kZnQsREM9Y29tP2NBQ2VydGlmawNhdGU\YmFzZT9vYmplY3RDdGFzcj1jZXJ0aWZpY2F0aW9uQXV0aG9y
aXR5MDSGA1UdEQQ0MDKgMAYKKwYBBAGCNxQCA6AIDCBhZG1pbmlzdHJhdG9yQGl1YWxpZGZ0MjcucjZGZ0LmNvbTANBgkqh
kiG9w0BAQsFAAOCAgEAp5ZEUsqw1\XH6oLedTwtQSDXraP5SprU6mKk+y5+W6osGicAxEwC183wwnmeXh1XRDJXRsx9U
yDsU3f5jJ94MMI7CR6mjLm88r9y8KxVoXikuBAka9+w2LsyxMunhQcd64JqK2lDCGJiEBti6R7+dZe4GRaDe9JpNPKoI4
RqCQ\TXc3knQ3MyGSbTktTtoliuaIGsmnKJ5LGG31tszI1vqvLuK+MavnUdLXKGeVcAGqYL6ZvinWOOJCFjggjEOhuOz
XsjzuPHMkHw0Ji6U8AEfnagQntXNGmEohVVEMFue0aRCmko9ragtFsfG1HXjSUoo5spGNOH9k4pmk4eanmJPGmCBB3DVC
gxjAYuIQvEnSV12Oydu6mOEuUy6oLfnKzIHWqmBqrjj2hAta+sNF\MSQqt2MVL8\G67F4W6xPfc+nGgh+1EDo+t6pPJ
UHyFog5CYQ+mRGerq2TcBq\Qv\MFwO3t2aEMroXdrW2EdNogHN25L8xrt37fd3s0+32h\Z8d7cmD5j9h7s7fUqH
dISg5U9b8UwFLH4ZATIGOSEaDP73XPLs7ic4rNJ88Y4e6LEK1UHCtBG0VNvdPHEVhctBKzhFZG0FI2kr0bfpudURymzxd
EHbExP4HErpgTLvcU7\S3AcMkz8DovXzG2CQnevAFkDpN8ne0yOraWwKE8Y="}
```

4.1.2 Server Response

```
HTTP/1.1 200 OK
Content-Length: 0
```

4.2 Getting Information about All Relying Party Trusts

4.2.1 Client Request

```
GET https://sts1.contoso.com/adfs/proxy/relyingpartytrusts?api-version=1 HTTP/1.1
Host: sts1.contoso.com
```

4.2.2 Server Response

```
HTTP/1.1 200 OK
Cache-Control: no-store
Pragma: no-cache
Content-Length: 469
Content-Type: application/json; charset=UTF-8
```

```
[{"enabled":true,"name":"Device Registration Service", "nonClaimsAware":false, "objectIdentifier":"4646dd08-49eb-e211-9867-00155d6ff01e", "publishedThroughProxy":false}, {"enabled":true, "name": "fedpassive", "nonClaimsAware":false, "objectIdentifier": "011ab67d-49eb-e211-9867-00155d6ff01e", "publishedThroughProxy":false}, {"enabled":true, "name": "integratedWindowsRp", "nonClaimsAware":true, "objectIdentifier": "071ab67d-49eb-e211-9867-00155d6ff01e", "publishedThroughProxy":true}]
```

4.3 Create a New Set of Published Settings on a Relying Party Trust

4.3.1 Client Request

```
POST https://sts1.contoso.com/adfs/proxy/relyingpartytrusts/7aeee25c-4beb-e211-9867-00155d6ff01e/fedpassive/publishedsettings?api-version=1 HTTP/1.1
Content-Type: application/json; charset=UTF-8
Host: sts1.contoso.com
Content-Length: 264
Expect: 100-continue

{"internalUrl":"https://urlInternal", "externalUrl":"https://urlExternal", "proxyTrustedEndpoint": "https://urlExternal"}
```

4.3.2 Server Response

```
HTTP/1.1 200 OK
Content-Length: 0
```

4.4 Remove an Existing Set of Published Settings on a Relying Party Trust

4.4.1 Client Request

```
DELETE https://sts1.contoso.com/adfs/proxy/relyingpartytrusts/0b153cca-4beb-e211-9867-00155d6ff01e/fedpassive/publishedsettings?api-version=1 HTTP/1.1
Content-Type: application/json; charset=UTF-8
Host: sts1.contoso.com
Content-Length: 155
Expect: 100-continue

{"externalUrl":"https://urlExternal", "proxyTrustedEndpoint":"https://urlExternal"}
```

4.4.2 Server Response

```
HTTP/1.1 200 OK
Content-Length: 0
```

4.5 Add a Key Value Pair to the Store

4.5.1 Client Request

```
POST https://sts1.contoso.com/adfs/proxy/webapplicationproxy/store/DLOWTTYDQMB2NAPRXFITNYKZXSVW8D7J0KCQEHOEA?api-version=1 HTTP/1.1
Content-Type: application/json; charset=UTF-8
Host: sts1.contoso.com
```

```
Content-Length: 33
Expect: 100-continue

{"value":"SOMEVALUE_THAT_I_HAVE"}
```

4.5.2 Server Response

```
HTTP/1.1 200 OK
Content-Length: 0
```

4.6 Retrieve a Value of a Key from the Store

4.6.1 Client Request

```
GET https://sts1.contoso.com/adfs/proxy/webapplicationproxy/store/MY_KEY?api-version=1
HTTP/1.1
Host: sts1.contoso.com
```

4.6.2 Server Response

```
HTTP/1.1 200 OK
Cache-Control: no-store
Pragma: no-cache
Content-Length: 60
Content-Type: application/json; charset=UTF-8

{"key":"MY_KEY","version":0,"value":"SOMEVALUE_THAT_I_HAVE"}
```

4.7 Update the Value of a Key Already in the Store

4.7.1 Client Request

```
PUT https://sts1.contoso.com/adfs/proxy/webapplicationproxy/store/MY_KEY?api-version=1
HTTP/1.1
Content-Type: application/json; charset=UTF-8
Host: sts1.contoso.com
Content-Length: 44
Expect: 100-continue

{"value":"ANOTHER VALUE___ NEW","version":0}
```

4.7.2 Server Response

```
HTTP/1.1 200 OK
Cache-Control: no-store
Pragma: no-cache
Content-Length: 28
Content-Type: application/json; charset=UTF-8

{"key":"MY_KEY","version":1}
```

4.8 Create a new Proxy Relying Party Trust

4.8.1 Client Request

```
POST https://sts1.contoso.com/adfs/proxy/webapplicationproxy/trust?api-version=1 HTTP/1.1
Content-Type: application/json; charset=UTF-8
Host: sts1.contoso.com
Content-Length: 35
Expect: 100-continue

{"Identifier":"https://\appProxy"}
```

4.8.2 Server Response

```
HTTP/1.1 200 OK
Content-Length: 0
```

4.9 Get the Proxy Relying Party Trust

4.9.1 Client Request

```
GET https://sts1.contoso.com/adfs/proxy/webapplicationproxy/trust?api-version=1 HTTP/1.1
Host: sts1.contoso.com
```

4.9.2 Server Response

```
HTTP/1.1 200 OK
Cache-Control: no-store
Pragma: no-cache
Content-Length: 35
Content-Type: application/json; charset=UTF-8

{"Identifier":"https://\appProxy"}
```


5 Security

5.1 Security Considerations for Implementers

None.

5.2 Index of Security Parameters

None.

6 Appendix A: Full JSON Schema

```
{
  "title" : "Proxy Trust",
  "type" : "object",
  "properties" :
  {
    "SerializedTrustCertificate" : {"type" : "string"}
  }
}

{
  "title" : "Proxy Trust Renewal",
  "type" : "object",
  "properties" :
  {
    "SerializedReplacementCertificate" : {"type" : "string"}
  }
}

{
  "title" : "Proxy Relying Party Trust",
  "type" : "object",
  "properties" :
  {
    "Identifier" : {"type" : "string"}
  }
}

{
  "title" : "Configuration",
  "type" : "object",
  "properties" :
  {
    "ServiceConfiguration" :
    {
      "type" : "object",
      "properties" :
      {
        "ServiceHostName" : {"type" : "string"},
        "HttpPort" : {"type" : "integer"},
        "HttpsPort" : {"type" : "integer"},
        "HttpsPortForUserTlsAuth" : {"type" : "integer"},
        "DeviceCertificateIssuers" :
        {
          "type" : "array",
          "items" : {"type" : "string"}
        },
        "ProxyTrustCertificateLifetime" : {"type" : "integer"},
        "DiscoveredUpnSuffixes" :
        {
          "type" : "array",
          "items" : {"type" : "string"}
        },
        "CustomUpnSuffixes" :
        {
          "type" : "array",
          "items" : {"type" : "string"}
        },
        "ServiceHostNameForUserTlsAuth" : {"type" : "string"}
      }
    },
    "EndpointConfiguration":{
      "type":"object",
      "properties":{
        "type":"object",
        "properties":{

```

```

        "Endpoints":{
        "type":"array",
        "items":[
        {
        "type":"object",
        "properties":{
        "Path":{"type":"string"},
        "PortType":{"enum":[0,1,2]},
        "AuthenticationSchemes":{"enum":[8,32768]},
        "ClientCertificateQueryMode":{"enum":[0,1,2]},
        "CertificateValidation":{"enum":[0,1,2]},
        "SupportsNtlm":{"type":"boolean"},
        "ServicePath":{"type":"string"},
        "ServicePortType":{"enum":[0,1,2]}
        }
        }
        ]
        }
    },
    "FarmBehavior" : {"type" : "string"},
    "IgnoreTokenBinding" : {"type" : "boolean"}
    "UpdatedFarmBehaviorLevel" : {"type" : "integer"},
}

{
    "title" : "Relying Party Trust List",
    "type" : "object",
    "properties" :
    {
        "relyingPartyTrustListArray" :
        {
            "type" : "array",
            "items" :
            {
                "type" : "object",
                "properties" :
                {
                    "objectIdentifier" : {"type" : "string"},
                    "name" : {"type" : "string"},
                    "publishedThroughProxy" : {"type" : "boolean"},
                    "nonClaimsAware" : {"type" : "boolean"},
                    "enabled" : {"type" : "boolean"}
                }
            }
        }
    }
}

{
    "title" : "Relying Party Trust",
    "type" : "object",
    "properties" :
    {
        "objectIdentifier" : {"type" : "string"},
        "name" : {"type" : "string"},
        "publishedThroughProxy" : {"type" : "boolean"},
        "nonClaimsAware" : {"type" : "boolean"},
        "enabled" : {"type" : "boolean"},
        "identifiers" :
        {
            "type" : "array",
            "items" : {"type" : "string"}
        },
        "proxyTrustedEndpoints" :
        {
            "type" : "array",

```

```

        "items" : {"type" : "string"}
    },
    "proxyEndpointMappings" :
    {
        "type" : "array",
        "items" :
        {
            "type" : "object",
            "properties" :
            {
                "Key" : {"type" : "string"},
                "Value" : {"type" : "string"}
            }
        }
    }
}

{
    "title" : "Relying Party Trust Publishing Settings",
    "type" : "object",
    "properties" :
    {
        "externalUrl" : {"type" : "string"},
        "internalUrl" : {"type" : "string"},
        "proxyTrustedEndpoint" : {"type" : "string"}
    }
}

{
    "title" : "Store Entry List",
    "type" : "object",
    "properties" :
    {
        "storeEntryListArray" :
        {
            "type" : "array",
            "items" : {"type" : "Store Entry"}
        }
    }
}

{
    "title" : "Store Entry",
    "type" : "object",
    "properties" :
    {
        "key" : {"type" : "string"},
        "version" : {"type" : "integer"},
        "value" : {"type" : "string"}
    }
}

{
    "title" : "Store Entry Key and Value",
    "type" : "object",
    "properties" :
    {
        "key" : {"type" : "string"},
        "value" : {"type" : "string"}
    }
}

{
    "title" : "Serialized Request with Certificate",
    "type" : "object",
    "properties" :
    {

```

```

"Request" :
{
  "type" : "object",
  "properties" :
  {
    "AcceptTypes" : {"type" : "string"},
    "Content" : [ <byte>, * ],
    "ContentEncoding" : {"type" : "string"},
    "ContentLength" : {"type" : "integer"},
    "ContentType" : {"type" : "string"},
    "Cookies" :
    {
      "type" : "object",
      "properties" :
      {
        "Name" : {"type" : "string"},
        "Value" : {"type" : "string"},
        "Path" : {"type" : "string"},
        "Domain" : {"type" : "string"},
        "Expires" : {"type" : "integer"},
        "Version" : {"type" : "integer"}
      }
    },
    "Headers" :
    {
      "type" : "array",
      "items" :
      {
        "type" : "object",
        "properties" :
        {
          "Name" : {"type" : "string"},
          "Value" : {"type" : "string"}
        }
      }
    },
    "HttpMethod" : {"type" : "string"},
    "RequestUri" : {"type" : "string"},
    "QueryString" : {"type" : "string"},
    "UserAgent" : {"type" : "string"},
    "UserHostAddress" : {"type" : "string"},
    "UserHostName" : {"type" : "string"},
    "UserLanguages" : {"type" : "string"}
  },
  "SerializedClientCertificate" : {"type" : "string"},
  "CertificateUsage" :
  {
    "enum" : [1, 2]
  },
  "ErrorType" :
  {
    "enum" : [0, 1]
  },
  "ErrorCode" : {"type" : "integer"}
}

{
  "title" : "Proxy Token",
  "type" : "object",
  "properties" :
  {
    "ver" : {"type" : "number"},
    "aud" : {"type" : "string"},
    "iat" : {"type" : "integer"},
    "exp" : {"type" : "integer"},
    "iss" : {"type" : "string"},
    "relyingpartytrustid" : {"type" : "string"},

```

```

    "deviceregid" : {"type" : "string"},
    "authinstant" : {"type" : "integer"},
    "authmethod" : {"type" : "string"},
    "upn" : {"type" : "string"}
  }
}

{
  "title" : "Combined Token",
  "type" : "object",
  "properties" :
  {
    "proxy_token" : {"type" : "Proxy Token"},
    "access_token" : {"type" : "string"}
  }
}

{
  "title" : "Proxy Token Wrapper",
  "type" : "object",
  "properties" :
  {
    "authToken" : {"type" : "Proxy Token"}
  }
}

{
  "title" : "Authentication Request",
  "type" : "object",
  "properties" :
  {
    "appRealm" : {"type" : " string"},
    "realm" : {"type" : " string"},
    "username" : {"type" : "string"},
    "password" : {"type" : "string"},
    "deviceCertificate" : {"type" : "string"},
    "userCertificate" : {"type" : "string"},

    "httpHeaders" :
    {
      "type" : "object",
      "properties" :
      {
        "Key" : {"type" : "string"},
        "Value" : {"type" : "string"}
      }
    }
  }
}

{
  "title" : "Error Response",
  "type" : "object",
  "properties" :
  {
    "id" : {"type" : "integer"},
    "message" : {"type" : "string"},
    "type" : {"type" : "string"}
  }
}

```

7 Appendix B: Product Behavior

The information in this specification is applicable to the following Microsoft products or supplemental software. References to product versions include updates to those products.

The terms "earlier" and "later", when used with a product version, refer to either all preceding versions or all subsequent versions, respectively. The term "through" refers to the inclusive range of versions. Applicable Microsoft products are listed chronologically in this section.

Windows Server

- Windows Server 2012 R2 operating system
- Windows Server 2016 operating system
- Windows Server operating system
- Windows Server 2019 operating system
- Windows Server 2022 operating system
- Windows Server 2025 operating system

Exceptions, if any, are noted in this section. If an update version, service pack or Knowledge Base (KB) number appears with a product name, the behavior changed in that update. The new behavior also applies to subsequent updates unless otherwise specified. If a product edition appears with the product version, behavior is different in that product edition.

Unless otherwise specified, any statement of optional behavior in this specification that is prescribed using the terms "SHOULD" or "SHOULD NOT" implies product behavior in accordance with the SHOULD or SHOULD NOT prescription. Unless otherwise specified, the term "MAY" implies that the product does not follow the prescription.

[<1> Section 2.2.1.5](#): The X-MS-ADFS-Proxy-Client-IP header is not sent by the **Web Application Proxy** on Windows Server 2012 R2.

[<2> Section 2.2.1.6](#): The X-MS-ProxyAuth-Token header is not sent by the Web Application Proxy on Windows Server 2012 R2 and later.

[<3> Section 2.2.2.4](#): The service-host-name-for-user-tls-auth field is not supported by the Web Application Proxy on Windows Server 2012 R2.

[<4> Section 2.2.2.4](#): The farm-behavior-version-number field is not supported by the Web Application Proxy on Windows Server 2012 R2.

[<5> Section 2.2.2.4](#): The ignore-token-binding field is not supported by the Web Application Proxy on Windows Server 2012 R2.

[<6> Section 2.2.2.4](#): The **updated-farm-behavior-level** field is not supported by the Web Application Proxy on Windows Server 2012 R2 and later.

[<7> Section 2.2.2.11](#): The Error-Type field of [Serialized Request with Certificate] is not supported on Windows Server 2012 R2. It is also not supported on Windows Server 2016 unless [\[MSKB-4034661\]](#) is installed.

[<8> Section 2.2.2.11](#): The Error-Code field of [Serialized Request with Certificate] is not supported on Windows Server 2012 R2. It is also not supported on Windows Server 2016 unless [\[MSKB-4034661\]](#) is installed.

[<9> Section 3.1.1.1](#): Any writes to [Server State] require, by default, 5 minutes to propagate to other nodes in the server in an **AD FS farm configuration** using WID.

[<10> Section 3.3.5.2.1.3](#): Windows does not remove the old certificate from [Server State].

[<11> Section 3.4.5](#): The following table shows the values of api-version that can be set by the Web Application Proxy in each operating system.

Operating System	api-version values supported
Windows Server 2012 R2	1
Windows Server 2016 and later	2

[<12> Section 3.11.5](#): In Windows Server 2012 R2, and in Windows Server 2016 without [MSKB-4034661] installed, the client simply ignores the request if no certificate was obtained.

[<13> Section 3.11.5.1](#): In Windows Server 2012 R2, and in Windows Server 2016 without [MSKB-4034661] installed, the client simply ignores a request with an invalid certificate.

[<14> Section 3.12.5.1.3](#): Windows validates that the sign-in request comes from a SAML-P IdP initiated request with a query string parameter RelayState containing an identifier of a web application in the server that relies on the WS-Fed protocol for authentication.

[<15> Section 3.12.5.1.5](#): Preauthentication for active clients is not supported on Windows Server 2012 R2.

[<16> Section 3.13.5.2.3](#): Preauthentication of active requests is not supported on Windows Server 2012 R2.

8 Change Tracking

This section identifies changes that were made to this document since the last release. Changes are classified as Major, Minor, or None.

The revision class **Major** means that the technical content in the document was significantly revised. Major changes affect protocol interoperability or implementation. Examples of major changes are:

- A document revision that incorporates changes to interoperability requirements.
- A document revision that captures changes to protocol functionality.

The revision class **Minor** means that the meaning of the technical content was clarified. Minor changes do not affect protocol interoperability or implementation. Examples of minor changes are updates to clarify ambiguity at the sentence, paragraph, or table level.

The revision class **None** means that no new technical changes were introduced. Minor editorial and formatting changes may have been made, but the relevant technical content is identical to the last released version.

The changes made to this document are listed in the following table. For more information, please contact dochelp@microsoft.com.

Section	Description	Revision class
2.2.2.4 Configuration	11887 : Added Endpoints field to JSON representation of EndpointConfiguration.	Major
2.2.2.7 Relying Party Trust Publishing Settings	11889 : Changed proxyTrustedEndpointUrl JSON keyword to proxyTrustedEndpoint.	Major
3.8.5.1.1.3 Processing Details	11889 : Changed proxyTrustedEndpointUrl JSON keyword to proxyTrustedEndpoint.	Major
3.8.5.1.2.3 Processing Details	11889 : Changed proxyTrustedEndpointUrl JSON keyword to proxyTrustedEndpoint.	Major
4.3.1 Client Request	11889 : Changed proxyTrustedEndpointUrl JSON keyword to proxyTrustedEndpoint.	Major
4.4.1 Client Request	11889 : Changed proxyTrustedEndpointUrl JSON keyword to proxyTrustedEndpoint.	Major
6 Appendix A: Full JSON Schema	11887 : Added array of Endpoints to the JSON schema.	Major
6 Appendix A: Full JSON Schema	11889 : Changed proxyTrustedEndpointUrl JSON keyword to proxyTrustedEndpoint.	Major

9 Index

A

[Applicability](#) 15
Application proxy runtime behaviors client
 [Abstract data model](#) 64
 [Initialization](#) 64
 [Message processing events and sequencing rules](#) 65
 [Other local events](#) 68
 [Timer events](#) 68
 [Timers](#) 64
Application proxy runtime behaviors server
 [Abstract data model](#) 60
 [Initialization](#) 60
 [Other local events](#) 64
 [Timer events](#) 64
 [Timers](#) 60
Application publishing client
 [Abstract data model](#) 54
 [Initialization](#) 54
 [Message processing events and sequencing rules](#) 54
 [Other local events](#) 55
 [Timer events](#) 55
 [Timers](#) 54
Application publishing server
 [Abstract data model](#) 51
 [Initialization](#) 51
 [Message processing events and sequencing rules](#) 51
 [Other local events](#) 54
 [Timer events](#) 54
 [Timers](#) 51

C

[Capability negotiation](#) 15
[Change tracking](#) 81
Common
 [Abstract data model](#) 28
 [Higher-layer triggered events](#) 29
 [Initialization](#) 29
 [Message processing events and sequencing rules](#) 29
 [Other local events](#) 29
 [Timer events](#) 29
 [Timers](#) 29
[Common data types](#) 16

F

[Fields - vendor-extensible](#) 15
[Full JSON schema](#) 74

G

[Glossary](#) 10

I

[Implementer - security considerations](#) 73

[Index of security parameters](#) 73
[Informative references](#) 13
[Introduction](#) 10

J

[JSON schema](#) 74

M

Messages
 [transport](#) 16

N

[Normative references](#) 11

O

[Overview \(synopsis\)](#) 13

P

[Parameters - security index](#) 73
[Preconditions](#) 15
[Prerequisites](#) 15
[Product behavior](#) 79
Proxy configuration client
 [Abstract data model](#) 49
 [Initialization](#) 49
 [Message processing events and sequencing rules](#) 49
 [Other local events](#) 51
 [Timer events](#) 51
 [Timers](#) 49
Proxy configuration server
 [Abstract data model](#) 43
 [Initialization](#) 43
 [Message processing events and sequencing rules](#) 43
 [Other local events](#) 48
 [Timer events](#) 48
 [Timers](#) 43
Proxy registration client
 [Abstract data model](#) 35
 [Higher-layer triggered events](#) 35
 [Initialization](#) 35
 [Message processing events and sequencing rules](#) 35
 [Other local events](#) 37
 [Timer events](#) 37
 [Timers](#) 35
Proxy registration server
 [Abstract data model](#) 29
 [Higher-layer triggered events](#) 30
 [Initialization](#) 30
 [Message processing events and sequencing rules](#) 30
 [Other local events](#) 35
 [Timer events](#) 35
 [Timers](#) 30

Proxy runtime behaviors client
[Abstract data model](#) 57
[Initialization](#) 58
[Message processing events and sequencing rules](#)
58
[Other local events](#) 60
[Timer events](#) 60
[Timers](#) 57
Proxy runtime behaviors server
[Abstract data model](#) 56
[Initialization](#) 56
[Message processing events and sequencing rules](#)
56
[Other local events](#) 57
[Timer events](#) 57
[Timers](#) 56

R

References
[informative](#) 13
[normative](#) 11
[Relationship to other protocols](#) 14

S

Security
[implementer considerations](#) 73
[parameter index](#) 73
Service configuration client
[Abstract data model](#) 41
[Initialization](#) 41
[Message processing events and sequencing rules](#)
42
[Other local events](#) 43
[Timer events](#) 43
[Timers](#) 41
Service configuration server
[Abstract data model](#) 38
[Initialization](#) 38
[Message processing events and sequencing rules](#)
38
[Other local events](#) 41
[Timer events](#) 41
[Timers](#) 38
[Standards assignments](#) 15

T

[Tracking changes](#) 81
[Transport](#) 16
[common data types](#) 16

V

[Vendor-extensible fields](#) 15
[Versioning](#) 15